

报告编号:9597AC895662468CBF67781753D3FFAD

送检文档:基于 ESP32 的低功耗触控智能手表系统设计与实现

作者:吴钟铤

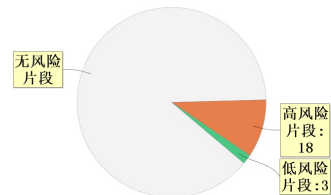
送检单位:河南理工大学

送检时间:2026-05-27 10:03:30

文本检测结果

疑似AIGC风险等级: **11.82% 低风险**疑似AIGC片段: **21**

疑似AIGC片段 = 高风险片段+中风险片段+低风险片段



图片检测结果

总图片数: 22张 疑似AI生成图占比: **0.0%**

疑似AI生成图0张=高风险0张+中风险0张+低风险0张 非疑似AI生成图: 22张

文本片段结果

序号	送检片段	疑似AIGC概率
1	在系统架构规划方面，为选取最契合本系统的软硬件实现方案，本文从主控芯片、触控屏、GUI库、视频解码、操作系统及固件升级等多个关键维度，对多种可行方案进行详细对比与选型分析：	高
2	方案C：NTP网络对时以及高精度外部RTC（SD3078）结合方案。系统在网络可用的情况下主动获取NTP时间并下发到外部RTC芯片进行校时并且写入时间库，在网络不可用或者断电后重新上电时，系统可以手动设置或使用RTC芯片保存的时间继续运行。	高
3	作为系统的物理基础，它控制底层外设，向上为应用层提供解耦、标准的驱动接口。主要包括六个最基本的硬件系统：触摸屏相关硬件（显示芯片、触控芯片、背光等）、存储硬件（SD卡、外置Flash芯片等）、总线与接口（SPI、I2C、GPIO底层外设等）、时钟与时序相关硬件（RTC实时时钟、系统定时器等）、网络相关硬件（Wi-Fi无线射频链路）、电源及唤醒相关硬件（供电、休眠唤醒等）。	高

序号	送检片段	疑似AIGC概率
4	作为系统的“后端”，是进行各种重要工作的地方，这里包含了设备的基本系统服务（例如时间同步、天气获取、Wi-Fi连接及状态管理、本地与云端OTA升级、功耗控制等），也包括多媒体以及娱乐相关的代码实现（如小说显示、图片解码、MJPEG视频解码、游戏的核心逻辑等）。同时还有异步队列用于文件扫描以及任务间的相互协作。本层的所有应用模块在完成对数据的操作或者状态发生变化之后不能直接调用图形库的方法，而应将状态变更的通知封装后放入异步传递的消息队列上报。	高
5	显示层内部逻辑包括界面路由交互、UI页面渲染以及底层显示输出深度三者联动：系统先统一截获用户触摸或按键等操作，以便完成各个不同页面间跳转（例如主表盘、系统设置、信息展示等）、状态传递及事件触发；之后利用UI渲染集合把所有业务数据都展示出来；最后由显示引擎负责组织整个图形绘制过程以及控制屏幕重绘时机，保证后端异步变化能及时准确地反映到前端屏幕上并且无撕裂、一致性良好。	高
6	如图2-2所示，系统实现了完善的低功耗状态管理。系统在一定时间内未检测到任何触控操作的情况下，系统将关闭背光，同时停止高能耗的WiFi以及音频视频等任务，让ESP32S3进入Light Sleep模式；而在硬件方面，采用具有自身休眠功能的CST816T触控IC，在有手指触摸或者移动显示屏时，CST816T会在很短时间拉低连接在主控芯片上的IO口触发外部中断，从而使ESP32S3立刻苏醒并恢复正常工作速度，做到“待机极省电、交互即刻反馈”。	高
7	如图2-5所示，系统使用A/B双分区进行固件更新，在升级过程中不会直接覆盖当前运行系统，而是先将新固件写入另一个分区；升级包来源于云端以及SD卡，但是操作方式是相同的，即先下载或者从SD卡中读取，然后写入并且验证其正确性，在确定无误之后再切换为目标分区并重启以进入新版本；设备启动之后还会进一步检测新固件是否正常可靠，如果存在问题就回退到旧版，这样既保证能够被更新又能尽可能减少由于升级导致的问题给设备带来的不便。	高
8	外设部分，芯片的所有GPIO都是可以引出的，方便以后的功能扩展；专门的SPI也可以引出用于控制高速显示或者存储设备，它对应的供电部分也有去耦电容，保证高速通信的信号质量良好。另外还留有调试接口，可以在编程时进行调试以及程序下载，便于开发和后期维护。	高
9	本设计使用一块1.83英寸显示触摸一体化模组，引脚定义见图3-6，整体示意图见3-7。其内部集成ST7789P3显示驱动电路和CST816T电容触摸检测电路，分辨率为240×284，具有高亮度、大视角和全方向可视等特点。相关图像采集与显示系统研究表明，TFT-LCD能够在嵌入式平台上完成实时图像显示，适合用于低成本、小尺寸人机交互终端[18]	高

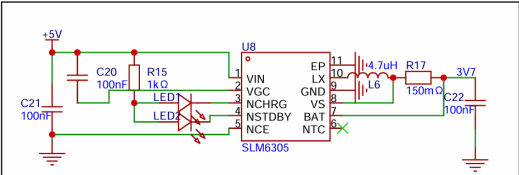
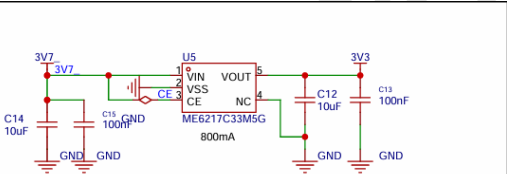
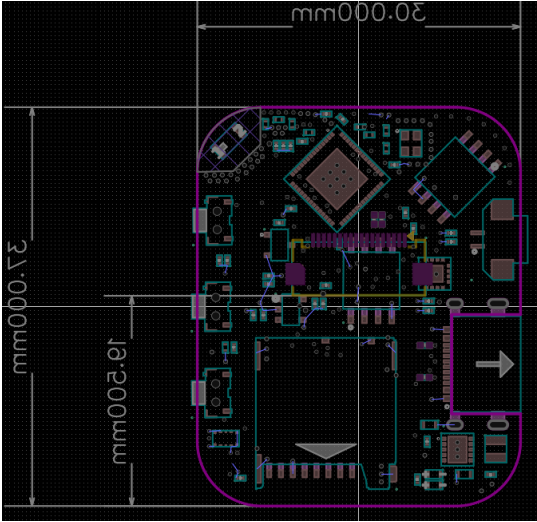
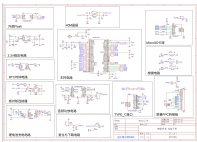
序号	送检片段	疑似AIGC概率
10	显示部分使用SPI硬件接口进行通信，由于其具有较高的传输速率以及较少占用硬件引脚的特点，使得整个系统更加简单明了。模组片选、命令/数据选择、串行时钟、串行数据均通过主控芯片上的SPI外设硬件引脚连接，而复位信号由主控芯片的普通IO口负责，用于上电初始化以及异常复位。背光控制信号接入主控芯片带有脉冲宽度调制输出功能的硬件引脚，可以用来设置屏幕的亮度。为了满足高分辨率下的良好刷新率，SPI有最大的80MHz时钟速度，在此基础上配合主控芯片内部的直接内存访问控制器，可以把显示缓冲区中的数据快速传输至显示驱动芯片中，大大减轻CPU负担并且提高传输速率，从而保证良好的用户体验。	高
11	从整个供电路径方面看，外接电源接入既可以给锂电池充电也可以经过一个稳压电路给整个设备供电，在没有外接电源的情况下，锂电池通过一个稳压芯片输出稳定的3.3V电压用于给整个电路供电。这个供电方式简单方便、供电稳定且充电安全可靠，有利于保证智能手表的正常运转。	低
12	应用层是整个系统的主体功能部分，主要有时间控制、小说阅读、图片查看、视频播放、网络连接、天气查询以及OTA更新等。为防止复杂的操作卡住UI线程，本系统采用FreeRTOS实时操作系统，在其中把文件读取、视频解码、网络通信、时间同步等都做成不同的任务来执行并且使用消息队列进行各个任务之间的通信。有文献报道FreeRTOS可以把一个嵌入式应用程序拆分成许多任务并为其提供合适的处理器时间以及其他系统的资源以保证系统的实时性和可靠性[7]。	高
13	显示层基于LVGL图形库实现界面显示、页面跳转、动画重绘以及触摸事件处理。应用服务层在接收到新的数据或者发生的状态改变时不会直接对屏幕进行操作，而是把天气结果、小说分页、视频一帧一帧地展示以及OTA下载进度等内容都组织成一条消息传递给显示任务，在显示层负责更新相应的界面内容以及状态变化。这样就使得业务代码和UI分离，有利于多个任务同时进行时屏幕刷新的安全性和一致性。	高
14	每个游戏模块都有自己的运行状态，例如角色位置，得分，卡牌状态或矩阵中的数值等；用户触控、拖动等操作作为输入事件影响这些状态的变化；同时界面根据当前状态进行重新绘制相应的画面以及得分等内容。这种方法简单高效并且占用较少资源，在小尺寸便携式设备上可顺畅运作。	低
15	总体来说，在ESP32-S3上实现表盘交互、资源读取、多媒体播放、文本浏览、网络连接和固件更新等功能，基本符合课题需求。之后可以针对SD卡读取速度、图片解码速度、视频帧刷新以及大负载情况下的任务调度进行改进提升系统的平滑性和实用性。	高
16	在系统运行状态下，LCD 屏幕背光与 Wi-Fi 射频模块是系统的两大核心耗电单元。测试过程中，以 10% 为步进调节屏幕亮度，并分别记录开启和关闭 Wi-Fi 时的功耗表现，具体测试数据如表 5-1 所示。	高

序号	送检片段	疑似AIGC概率
17	结合450mAh电池容量估算，在10%亮度、WiFi关闭的情况下，设备理论亮屏待机时间为约7.7小时；而在100%亮度、WiFi关闭情况下，则理论亮屏待机时间为约2.7小时。	低
18	系统设有两级低功耗模式：约120秒未进行任何操作后，系统会自动进入LightSleep轻度休眠；约3秒长按确认键之后，系统会进入DeepSleep深度休眠。这两种模式都是为了解决不同时间段内使用的需要而设计，在保证较低能耗的基础上也考虑到了唤醒以及方便性问题。	高
19	之后进一步优化可考虑引出LCD屏幕TE同步信号并连接到ESP32-S3外部中断，然后在LVGL刷新过程中加入垂直同步处理，让DMA刷屏尽量避开屏幕刷新区域。这可以在一定程度上减少快速移动的画面出现抖动的概率，在播放视频或者拖拽菜单等情况下可以使显示更加稳定。	高
20	这两种休眠方式都有各自的功能。LightSleep是在短时间内休眠，仍然保留触摸或按键唤醒的功能，能够快速进行人机交互；而DeepSleep是长时间休眠，依靠物理按键唤醒以减少误操作的可能性，在口袋或包中不易被触碰从而避免意外唤醒。后续工作不应仅仅是减少单一方面的功耗，还应、唤醒时间和续航等方面综合考虑整体设计的问题。	高
21	总体而言，目前系统已实现功能闭环以及样机测试，在显示同步、存储带宽、低功耗硬件断电及小型化等方面还有待提高，在接下来的工作中可以针对“更平滑显示、更低待机电耗、更小体积”进行改进以使系统更加稳定可靠并具有一定的商品化前景。	高

疑似AI生成图结果

序号	疑似AI生成图	疑似AI生成概率
1		0.0% 非疑似AI生成图
2		0.0% 非疑似AI生成图
3		0.0%

序号	疑似AI生成图	疑似AI生成概率
		非疑似AI生成图
4		0.0% 非疑似AI生成图
5		0.0% 非疑似AI生成图
6		0.0% 非疑似AI生成图
7		0.0% 非疑似AI生成图
8		0.0% 非疑似AI生成图
9		0.0%

序号	疑似AI生成图	疑似AI生成概率
		非疑似AI生成图
10		0.0% 非疑似AI生成图
11	 锂电池充电电路	0.0% 非疑似AI生成图
12	 3.3V稳压电路	0.0% 非疑似AI生成图
13		0.0% 非疑似AI生成图
14		0.0% 非疑似AI生成图

序号	疑似AI生成图	疑似AI生成概率
15		0.0% 非疑似AI生成图
16		0.0% 非疑似AI生成图
17		0.0% 非疑似AI生成图

序号	疑似AI生成图	疑似AI生成概率
18	系统正常运行 初始化低功耗管理任务并记录最近一次用户活动时间 周期检测空闲时间并监听确认键长按状态 确认键是否长按 3 秒? 是 否 空闲时间是否超过 2 分钟? 是 否 自动轻度休眠路径 通知显示任务暂停刷新并保存当前界面状态 是 否 视频是否正在播放? 是 否 停止播放并记录返回位置 关闭屏幕背光和 LCD 显示 记录 WiFi 状态并临时关闭无线连接 保持 RTC 等必要外设运行 配置按键和触摸作为唤醒源进入 LightSleep 否 是 是否检测到唤醒事件? 是 否 重新初始化触摸、LCD 和背光 按需恢复 WiFi 连接并恢复 LVGL 界面运行 快速恢复到交互状态 强制深度休眠路径 确认键长按后进入深度休眠准备流程 通知显示任务暂停并停止视频等高功耗业务 保存文件阅读进度并关闭 SD 卡相关操作 关闭屏幕、WiFi、I2C 和相关外设 等待确认键释放避免立即重复唤醒 配置 GPIO7 为 RTC 唤醒引脚并保持必要 RTC 电平 进入 DeepSleep 深度休眠 是否再次按下确认键? 是 否 硬件唤醒并重新启动系统 从启动流程重新运行	0.0%
19		0.0%
20		0.0%

序号	疑似AI生成图	疑似AI生成概率
21		0.0% 非疑似AI生成图
22		0.0% 非疑似AI生成图

全文标红报告

河南理工大学

本科毕业论文（设计）
题目 基于ESP32的低功耗触控智能手表系统设计与实现
学院名称 电气工程与自动化学院
专业名称 自动化
年级班级 自动化2202
学生姓名 吴钟铤
指导教师 乔美英
2026年 5月

摘要

伴随着物联网快速发展，在嵌入式开发领域同时拥有良好人机交互体验、强大多媒体处理能力和低功耗以及能够连接云服务器的智能手表受到广泛关注。

但是目前的智能手表方案，在硬件资源有限的嵌入式平台上，仍然存在图形界面交互不够流畅的问题、缺乏多媒体处理能力和难以进行有效功耗管理以及难以实现良好的云服务集成等难题。本文提出一种基于ESP32-S3和LVGL开源图形库的低功耗触控智能手表设计方案，旨在资源有限情况下开发出功能完善、使用方便穿戴设备。

在硬件设计上，此系统采用ESP32-S3R8作为主控，配备一块1.83寸、分辨率为240x284触摸屏。这块屏幕是ST7789通过SPI与主控进行通信从而实现快速刷新显示；而触摸则是采用CST816T电容触摸屏，使用标准I2C接口配合外部中断实现快速响应，另外它功耗很低只有8.0μA，这对整个设备工作时间有很大帮助。此外，ESP32-S3R8还连接着一块W25Q128闪存，16MB外部Flash用于存放跳转固件、字体和图片等用于系统应用与便于后期对系统升级扩展；此外还使用SD卡存储单元和SD3078实时时钟来满足大量多媒体文件读取写入以及准确计时的需求。

在软件设计上，系统基于ESP-IDF框架以及FreeRTOS实时操作系统进行模块化开发。使用NXP提供的图形界面设计工具GUI Guider完成UI绘制工作，在底层进行深度裁剪LVGL并启用双缓冲使显示更加平滑流畅的手表界面、层级菜单、全键盘输入以及各种小游戏。考虑到单片机资源限制，内部移植MJPEG视频播放器的解码文件，并且配合一个特别设计异步文件读取操作，在读取大文件过程中不会导致UI卡顿。ESP32-S3R8自定义了一套WiFi管理库，在此之上实现了通过NTP进行网络校时并通过知心天气网站下拉天气数据获取当前天气；同时利用MQTT协议接

入中国移动OneNet物联云平台，可以发送设备信息并且远程交互。还有本地固件更新以及通过OneNet进行OTA空中升级功能，并且配合低功耗功能让设备长时间正常工作。

根据系统综合调试，该智能手表网络连接良好，触摸屏反应灵敏，播放MJPEG视频与PNG图像流畅，OneNet云端通信正常，在各个方面都满足了设计要求。本课题完成了一个较高集成度穿戴设备软件硬件设计方案的同时也给在嵌入式平台上进行轻量级多媒体处理以及整个物联网穿戴设备开发提供一个可供借鉴的实际工程案例。

关键词：ESP32-S3；LVGL 图形库；智能手表；MJPEG 视频解码；OTA 远程升级

Abstract

Due to the blistering growth of the Internet of Things, smartwatches have gained considerable interest in the sphere of embedded development with their ability to have a smooth user experience, strong multimedia processing, low power usage, and connection to cloud servers. Nevertheless, existing smartwatch applications continue to be plagued by various problems including lack of fluidity when using graphical interfaces, poor multimedia processing ability, inability to effectively manage power consumption and problem of integrating the clouds services smoothly on embedded devices with small hardware resources. The proposed design scheme in this paper is a low-power touchscreen smartwatch design scheme that relies on the ESP32-S3 and an open-source graphics library, LVGL, to create useful and user-friendly wearable devices despite limited resources.

In terms of hardware, the whole system runs on ESP32-S3R8 as the main controller, which has a 1.83-inch, 240 x 284-pixel capacitance touchscreen. The ST7789P3 screen is displayed in a fast way using SPI; the touch screen is CST816T capacitive touch screen. Since it is compatible with the standard I2C interface and the external interrupt, it may have almost no delay response and the idle current is very low at 8.0uA which would help prolong the time the machine runs. Simultaneously, ESP32-S3R8 is also connected to a W25Q128, 16 MB external flash to save information (jump firmware, fonts, and pictures) to enable future system upgrades and enhance image quality ; and moreover, the system also has an SD card storage unit and SD3078 real-time clock to satisfy the demands of reading and writing large quantities of multimedia files and to guarantee the preciseness of timing.

In terms of software design and engineering implementation, the project was implemented using the ESP-IDF framework and FreeRTOS real-time operating system. The UI was done with the help of the GUI Guider tool offered by NXP. LVGL was highly customized in the lower layers and dual buffering was introduced to provide a smoother performance of the display. Hierarchical menus, the watch interface, full keyboard input, and other games were implemented. Due to the limited resources of the microcontroller, the hardware-accelerated PNG decoder and MJPEG video player were coded in-house and the asynchronous file read mechanism was specifically created to avoid UI lag during large file reads. The ESP32-S3R8 was modified with a WiFi management library that allowed synchronizing the time on the network with NTP and obtaining the current weather data. It also made use of the MQTT protocol to communicate with China Mobile OneNet IoT cloud platform to transmit the information of the device and implement control remotely. The project also had local firmware upgrades and OneNet OTA firmware upgrades, as well as low power algorithms enabling the device to operate continuously over a long period.

Overall, the smartwatch has good network connectivity, a responsive touchscreen, smooth playback of MJPEG videos, normal cloud communication via OneNet, and meets the design requirements in all aspects. This project not only completed a high-integrated wearable device software-hardware design scheme but also provided a practical engineering case study for lightweight multimedia processing on embedded platforms and full-process development of the Internet of Things.

Keywords : ESP32-S3 ; LVGL graphics library ; smart watch ; MJPEG ;OTA

目录

1.绪论1

1.1 课题背景及研究的目的	1
1.2 低功耗手表领域的国内外研究现状及分析	1
1.3 ESP32-S3R8低功耗手表研究的主要内容	2
2.低功耗手表的需求分析与总体设计	5
2.1 需求分析	5
2.2 方案比较	7
2.3 ESP32低功耗手表总体设计	9
2.3.1 硬件架构设计	9
2.3.2.软件整体框架	10
2.3.3 软件局部框架	11
2.4 本章小结	14
3.硬件设计	15
3.1 ESP32-S3 最小系统电路设计	15
3.1.1.核心主控	15
3.1.2外部Flash存储电路	16
3.1.3 40MHz晶振时钟电路	17
3.1.4 下载控制与复位电路	18
3.2显示与触控模组电路设计	19
3.2.1 LCD 显示接口电路 (ST7789P3)	19
3.2.2 触控接口电路 (CST816T)	19
3.3 独立 RTC 时钟电路设计	19
3.4外部大容量存储电路设计 (Micro SD 卡)	21
3.5 电源管理与充电电路设计	21
3.6 本章小结	22
4.软件设计	25
4.1 低功耗手表系统整体架构与多任务协同调度设计	25
4.2 横向架构的核心业务服务模块设计	26
4.2.1 SD卡综合应用与设计	26
4.2.2 时钟管理与时间协同模块设计	27
4.2.3 OTA远程升级与固件维护模块设计	29
4.2.4 开源休闲游戏扩展设计	31
4.2.5 系统多级低功耗电源调度机制设计	31
4.3本章小结	33
5.低功耗手表系统综合调试	35
5.1 硬件系统实物构建与整机组装集成	35
5.1.1 核心控制板贴装与焊接工艺验证	35
5.1.2 机械结构建模与3D打印成型	36
5.1.3 整机集成与上电初步验证	37
5.2 核心业务功能与整机交互体验评估	38
5.3 系统功耗与多级休眠机制测试	39
5.3.1 动态功耗测试结果分析	39
5.3.2 多级休眠功耗异常分析与模式优势探讨	41
5.4系统现存不足与未来优化展望	41
5.4.1 软件渲染与总线带宽瓶颈分析	41
5.4.2 硬件电源与低功耗设计改进	42
5.4.3 结构工程与整机空间优化	42

5.5 本章小结43

6.结论45

致谢47

参考文献49

1.绪论

1.1 课题背景及研究的目的

目前,在物联网以及微电子技术不断发展背景下,可穿戴设备已经成为移动互联网和个人信息服务重要工具。而智能手表也从最初简单的计时器发展成为可以实现信息通信、视频播放等功能个人智能设备。据相关报道显示,可穿戴设备对于健康监控、健身跟踪及联网功能有着良好的应用前景[1]。但是在嵌入式设备中,开发出一款廉价且具有良好视觉效果并具有较长续航时间智能手表仍然存在一些困难。传统的低功耗智能手环或入门级RTOS(实时操作系统)手表因为主控处理性能较低,所以它们的图形化界面非常粗糙,也没有多媒体播放能力和复杂交互功能。而像苹果公司生产的Apple Watch或谷歌的Wear OS这样的高端智能手表虽然强大但价格昂贵。因此,在资源有限嵌入式微控制器上研制出一款功能全面、运行平滑并且能耗低可穿戴设备是目前嵌入式物联网领域一个重要课题。本文主要围绕如何基于ESP32-S3R8微控制器以及LVGL图形库设计并实现一款低功耗触控手表进行探讨。通过发挥ESP32-S3R8的双核优势,结合小型化实时操作系统和最新图形技术克服传统单片机在多媒体处理和人机交互方面局限性。本文旨在为轻量级物联网穿戴设备开发提供一个经济实惠、高度集成软硬件方案,说明在MCU上也是可以做到视频软解码、丰富娱乐功能以及良好云端服务等,为实际应用提供一定指导意义。

1.2 低功耗手表领域的国内外研究现状及分析

在全球可穿戴设备市场中,智能手表研究及产业化十分火热,在国外,高档智能手表主要被大牌企业所垄断,这些产品一般使用性能优越的处理器并装有高度定制的操作系统,在技术和生态上拥有明显优势,但是其系统的复杂性、高昂的研发费用以及高昂的成本价格并不适合用于低成本嵌入式实践或工程验证。而在国内,智能穿戴行业发展较快,在工艺水平、成本以及个性化设计等方面都具有一定竞争力。对于中低端低功耗智能穿戴设备,行业内一般使用杰理、鸿芯等低成本芯片,这些芯片利用裸机循环配合消息队列以及自定义GUI框架实现基本功能后再由工程师进行二次开发,以达到大规模量产的目的。已有文献表明,嵌入式平台可以良好地支持低功耗通信、触摸操作以及信息读取等[2]。相比于传统的裸机方式,RTOS加上轻量级GUI框架具有更好的及时性、并发性和用户体验;而本文正是在此基础上完成整个系统的开发以及测试工作,给类似的产品开发提供了一种可以借鉴的方法。

从具体的技术路径上来说,由于ESP32系列具有内置Wi-Fi功能、价格低廉并且开发环境良好,已经被大量运用到小型物联网设备上[3]。另外轻量级图形库如LVGL可在资源受限的微控制器上绘制复杂的图形化界面,也为嵌入式GUI开发提供了新思路。近年来也有研究人员开始关注便携式智能设备上低功耗管理、人机交互以及续航能力等。但是将双缓冲渲染、SD卡视频流硬件加速解码、多任务并行异步存取、基于MQTT协议的一体化云和固件更新结合在一个小巧便携式设备上还是很少见。

1.3 ESP32-S3R8低功耗手表研究的主要内容

本论文以基于ESP32-S3R8低功耗触控智能手表软硬件设计与实现为主题,主要对系统的总体方案、硬件电路设计、软件功能实现、系统调试测试等进行阐述与设计。本文的主要工作包括以下几个方面:

对低功耗触控智能手表的需求及总体方案的设计,在结合智能手表的应用环境的基础上,探讨了该设备应具有触摸操作、图形界面显示、多媒体播放、文件存储、网络通信、时间同步、天气获取、云平台接入、OTA固件升级以及低功耗等基本要求;其次从上述需求出发,对主控芯片、显示屏、触摸屏、外部Flash、SD卡、RTC实时时钟、供电和无线通信方式进行了方案选型规划。

接下来介绍了智能手表硬件系统的开发。硬件方面是以ESP32-S3R8为主控芯片,在此基础上实现了基本电路、显示屏及触摸屏、外部存储器、RTC时钟、SD卡、供电管理等。其中,显示使用ST7789P3驱动的1.83寸电容触摸屏并通过SPI进行图像数据传输;触摸是用CST816T触摸屏控制器并通过I2C读取触摸数据;外部Flash是W25Q128用来存放字体、图片以及其他信息;SD卡用于保存照片、文字、影片等多媒体资料;RTC用于在无网络情况下维持时间准确性。以上所有硬件配合使用,给后面软件部分工作打下坚实的基础。

后面主要完成智能手表软件设计。软件方面采用ESP-IDF开发框架以及FreeRTOS实时操作系统进行模块化设计,包括显示驱动、触摸输入、LVGL图形界面、文件系统、多媒体播放、网络通信、云平台接入、OTA升级和低功耗管理等功能模块,在图形界面部分使用GUI Guider进行界面布局设计并用LVGL实现手表主界面、菜单界面、设

置界面、文件浏览界面及相关页面，在多媒体部分着重实现SD卡读取文件、PNG图片展示、TXT文本阅读及MJPEG视频播放，并采用异步文件读取以及任务调度降低大量文件读取对界面刷新影响，在网络通信部分实现WiFi连接、NTP时间同步、天气信息获取以及MQTT通信，让手表可以连接到OneNet物联网云平台，在系统维护中实现本地SD卡升级与云端OTA升级两种方式，另外还加入了屏幕亮度调节、睡眠管理和外设供电管理以达到省电效果。

再后来主要是对整个系统的调通及各项功能实现情况。首先是对硬件焊接与整机组装、显示与触摸功能测试、文件系统读写测试、多媒体播放测试、WiFi网络连接测试、OneNet云平台通信测试、OTA升级测试和低功耗运行测试等进行详细说明，在此基础上对各部分的功能进行检查确认其连接正常并且软件各项功能能够正常使用；其次针对在测试中遇到的问题如显示刷新卡顿、文件读取延迟等问题进行原因分析并加以解决从而使得系统运行更加流畅可靠并且给用户带来更好的使用感受。

最后对全文工作的概括与展望，如ESP32-S3R8智能手表硬件设计、LVGL图形界面开发、多媒体文件处理、物联网通信、OTA升级、低功耗管理等。其次指出现有的问题，如整机结构紧密程度、进一步降低功耗、界面美观性、多媒体播放流畅度以及云平台的功能拓展等方面的问题。最后提出未来的研究方向，以便更好地改进和完善低功耗智能手表系统。

2.低功耗手表的需求分析与总体设计

2.1 需求分析

本课题研究一种具有信息展示以及多媒体娱乐功能的低功耗智能手表。市场上已有的智能可穿戴设备显示，智能手表一般都需要具备触摸操作、低功耗通信、任务管理以及屏幕显示等部分[4]。结合可穿戴设备使用场景及项目目标，本文所提出智能手表系统需要满足以下几点需求。

(1)高清图形渲染与人机交互需求

智能手表清晰显示与方便快捷操作是其价值所在。以LVGL图形库为基础开发全彩、可触控显示屏幕，实现具有多层次菜单选择、全部字母键盘输入、平滑滚动效果以及多个系统设置等功能。并且要有丰富有趣开源小游戏供用户选择，例如贪吃蛇、2048等。

(2)多媒体解码与海量数据存储需求

为了使传统的手表不再只实现单一的功能，低功耗手表系统需要支持大容量SD卡的数据本地读写操作，在此基础上还需要能够挂载SD卡上的文件系统并在此之上实现多媒体应用，如能够查看本地的NPG图像，能够阅读具有阅读进度保存功能的TXT格式的电子书，以及可以进行MJPEG视频流的软件解码并且可以流畅播放。

(3)物联网协同与云端通信需求

作为现代物联网穿戴设备，该系统应具有联网能力，在其内部配备WiFi连接管理器并通过MQTT协议可靠地连接到中国移动OneNET物联网云平台进行设备间通信以及OTA固件更新等后期工作。基于此，一些研究者对ESP32物联网系统的运用进行了探讨发现ESP32利用WiFi通讯及OneNET云平台可实现低成本、稳定的通信及远程控制[5]。

(4) 时间管理需求与基础天气环境获取

作为手表系统首先要有准确的时间显示的功能。在硬件方面需要有独立的RTC（例如SD3078）实时时钟来保证从低功耗状态恢复仍然能够正常工作，在软件方面要支持基于WiFi的NTP网络对时功能，从NTP获取准确的时间会保存到芯片的时间time库以及实时时钟芯片中。另外还需要具有监测周围环境的能力，可以通过发送HTTP请求访问网络上的API，获取并解析出本地当前的天气情况及温度等。

(5) 系统维护与电源管理需求

由于智能手表对续航有极高的要求，系统需要有很强的节能能力，可以被动地进入light sleep休眠状态并且可以通过触摸屏打断而从睡眠中被唤醒；也可以主动地进入deep sleep休眠状态并且可以通过把确认键设置为一个GPIO来唤醒。另外考虑到后期维护成本的问题，固件升级也是必要的，且要同时支持通过SD卡本地升级以及通过服务器下发进行OTA升级。

而在性能指标分析方面，为确保手表在实际穿戴场景下的稳定性和流畅性，本系统需满足以下核心性能指标：

屏幕显示与视频播放指标

系统界面对表盘显示、主菜单滑动、页面切换以及快捷面板弹出时应顺畅无延迟，不能有较为明显的卡顿现象。显示刷新使用LVGL图形库以及SPI-DMA搬运进行，界面状态由消息队列进行更新，以减少业务线程对屏幕进行控制而造成阻塞与卡死的情况发生。对于MJPEG视频播放，视频播放平均帧率为25FPS左右，可以保证一定的连贯性

，在较高负载下可以有轻微的掉帧情况，但是不影响正常观看。

（2）触控响应与手势识别指标

系统要使触摸输入有较好的即时性。电容触控芯片检测到点击或滑动时，程序需立即获取其位置信息并交给LVGL事件处理来处理相应页面的操作。一般常见的点击、上下左右滑动等操作的响应时间需小于50ms，使用户在浏览菜单、返回上一级页面、呼出快捷面板等情况下能感受到明显的反馈。手势可识别上滑、下滑、左滑以及右滑并且可以触发表盘、主菜单、快捷面板以及各个选项页等之间的转换。

（3）功耗与休眠控制指标

系统需有分级低功耗管理功能，在正常使用下，屏幕背光亮度可按需进行调节从而实现亮屏状态下省电的目的；WiFi模块一般情况下不会一直处于较高功率的状态，只会在需要进行网络对时、天气更新或是OTA升级时才会短暂地打开。当设备连续大约120秒没有任何操作时，会自动进入到LightSleep轻度休眠状态，在这种状态下关闭屏幕背光以及一些外围设备但是保留触摸或者按键唤醒的能力。如果用户长按确认键约三秒钟，则会进入DeepSleep深度休眠状态，在这种状态下只保留最基本的物理按键唤醒的能力来防止长时间存放造成误唤醒。

（4）网络连接与通信稳定性指标

系统需具备WiFi连接、断开检测以及自动重连功能，在弱网或者短时间内掉线的情况下，程序需要能够自动尝试重新连接，重连次数不少于6次并且有相应的提示信息显示当前连接状态。当网络连接成功之后，系统可以实现NTP对时、获取天气信息及与OneNET进行交互等操作。同时MQTT通信也要保证对其主题订阅、属性上报以及OTA通知接收的可靠性，在云上发布更新指令后设备能立即作出反应。

（5）OTA升级与固件维护指标

系统需支持远程OTA升级以及SD卡本地固件升级两种方法，在升级期间新固件不应写入当前正在运行分区，防止直接覆盖正在运行系统而使设备无法启动。已有嵌入式软件A/B分区研究显示双分区有利于提升系统在进行固件升级时稳定性并且当出现故障时还可以有备用分区可以使用[6]。固件写入应当有进度条显示、异常中断能力和完成校验功能；写入完毕之后必须经过校验后才能切换到新启动分区。如果升级失败或者校验不过关，则保持原有固件可用以减少由于升级造成设备无法启动几率。整个OTA过程需具备提示用户操作、可取消、可回退以及可重新升级能力。

2.2 方案比较

在系统架构规划方面，为选取最契合本系统的软硬件实现方案，本文从主控芯片、触控屏、GUI库、视频解码、操作系统及固件升级等多个关键维度，对多种可行方案进行详细对比与选型分析：

（1）主控芯片方案选择

方案 A：使用性能优越的传统单片机（例如STM32F4/H7系列）。此方案工业级稳定可靠，外设资源充足。但是需要添加物联网功能时，则需外接如ESP8266等WiFi模块，这样会占用较大的PCB面积及增加硬件成本，另外两个芯片之间通信也会导致系统功耗增大，不利于穿戴设备小型化设计。

方案 B：使用应用级处理器（如NXP或全志芯片）。主频很高，可以运行Linux系统。但是这种方案功耗很大，需要大容量电池支持，而且硬件实现非常复杂，需要外接高速DDR内存，不适合本项目“低功耗嵌入式手表”。

方案 C：使用乐鑫 ESP32-S3 微控制器。这是一款双核 Xtensa LX7 芯片，主频达到 240MHz，原生集成 2.4GHz Wi-Fi，在此基础上还提供专门用于AI计算以及图像处理的向量指令集，非常适合进行 JPEG 图像解码工作。

总体而言：方案三（ESP32-S3）在算力、集成度、无线协同以及低功耗上都表现优秀，所以作为本系统主控芯片。

（2）显示屏与触控模组方案选择

方案 A：使用并口（8080/RGB 接口）TFT LCD 屏。显示刷新速度快但是需占用十多至二十多个 GPIO 引脚，使主控芯片引脚紧缺不能连接其他传感器。

方案 B：使用使用I2C接口的低速OLED屏。该屏优点是功耗小、对比度高，但是并不支持触摸操作并且市场上常见的读写速率以及分辨率不能适应复杂的UI以及视频等多媒体内容显示。

方案三：使用1.83英寸ST7789P3驱动LCD屏以及CST816T触摸屏。从屏方面看，用4线SPI总线进行通讯，最大时钟速度为80MHz，只需四个引脚就可以达到很高的传输速率；在触屏方面，用I2C接口连接，大大降低主控的工作压力。

综合评估：方案C在引脚占用率以及显示效果方面良好，确定为本项目显示触控方案。

(3) 图形界面 (GUI)开发框架方案选择

方案 A：使用裸机的手写 GUI 框架。不依赖任何第三方库，自己编写打点函数来画出一个界面。此方法开发效率低下，代码耦合性较强并且不能做到抗锯齿以及透明度动画等功能。

方案 B：使用 emWin图形库，这个平台在工业上被大量使用，但是大多数都有版权的问题，而且现代的 UI 组件少，开发起来比较僵硬。

方案三：使用LVGL开源图形库。LVGL是目前嵌入式领域最火的一款免费开源GUI框架，具有众多控件例如滚轮、进度条、键盘等，支持硬件双缓冲以及DMA加速，并且可以借助强大的NXP GUI Guider可视化拖拽开发工具。

综合评价：方案C无论是在显示效果上还是在开发效率上都是最佳，用作本系统图形引擎。

(4) 视频解码与播放方案选择

方案 A：使用H.264/H.265软解码。因为ESP32-S3无视频解码硬件加速器，仅用CPU进行H.264软解码会导致帧率极低甚至造成系统崩溃，不可取。

方案 B：使用未经压缩的RGB565图片帧依次显示。此方案几乎无需CPU进行解码计算开销，但是每一帧大小为240×RGB×284即约272KB大小的一幅图片，在一分钟视频中就有接近250MB之巨，而且由于SD卡SPI读取速率过低导致视频播放极其缓慢甚至不能正常播放视频。

方案三：使用MJPEG视频流方式。将视频转化为一个个压缩的JPEG图片，ESP32-S3通过硬件解码库对JPEG帧进行快速解压缩的同时也大大减轻了SD卡的读取负担。

综合评价：考虑主控特点，方案C是最适合在嵌入式平台上进行视频播放。

(5) 操作系统与任务调度方案选择

方案 A：使用传统的前后台系统（裸机循环）不需要给它分配单独的任务栈空间，占用内存极小但是由于本系统有Wi-Fi网络通信、LVGL显示、视频解码以及外设轮询等多种功能，在while(1)的大循环中依次进行工作，很容易造成各个任务之间互相等待从而根本无法达到音频视频还有网络交互的效果。

方案B：使用FreeRTOS实时操作系统。乐鑫官方的ESP-IDF开发工具链对底层进行了大量优化，基于FreeRTOS，提供丰富的信号量、互斥锁、消息队列等并发处理方式，可轻松分离高负载的视频解码工作以及UI更新的工作。

综上所述：方案 C 在系统的原生兼容性、多核使用率及多媒体实时调度上最优，是作为本系统底层操作系统的平台。

(6) 设备授时与时钟同步方案选择

方案 A：纯硬件 RTC 芯片（例如 DS3078）。利用外部时钟芯片实现计时，在无网络情况下可以正常工作，但是由于存在晶振温漂等问题，长时间使用会出现很大的误差（时钟漂移），并且第一次上电需要人为设置时间，之后每过一段时间又要重新调整时间，用户体验较差。

方案B：纯网络SNTP授时。设备连接上Wi-Fi之后，向互联网上的NTP时间服务器请求高精度UNIX时间戳。此方法时间准确，但缺点是如果网络中断或者设备处于低功耗模式下被唤醒时，如果没有网络连接，则会失去当前时间，这与手表脱离网络使用的要求不符。

方案C：NTP网络对时以及高精度外部RTC（SD3078）结合方案。系统在网络可用的情况下主动获取NTP时间并下发到外部RTC芯片进行校时并且写入时间库，在网络不可用或者断电后重新上电时，系统可以手动设置或使用RTC芯片保存的时间继续运行。

总结：方案C实现较高精度以及全天候高可靠性良好结合，在设备断网或掉电时不会出现时间丢失情况，最终确定为本系统时钟机制。

(7) 固件升级安全方案选择

方案A：单分区直接覆盖式OTA，虽然可以做到无线更新，但是下载新固件会直接替换正在运行的应用程序，在线或者断电时会造成系统无法启动的风险，容错率为0。

方案 B：基于 A/B 双分区云/本地 SD 卡双通道固件更新方式。在Flash中划分出独立的两个分区，在新固件到来时下载到后台备用分区，下载完成之后进行校验，如果新固件校验通过，则跳转；如果新的固件校验失败或者出现异常情况，Bootloader引导程序将自动回退到之前的稳定分区。

总结：方案B具有很高的工业级安全性，在后期对设备进行维护以及功能上的更新都能保证其稳定可靠运行，作

为我司升级系统的选择。

2.3 ESP32低功耗手表总体设计

2.3.1 硬件架构设计

本系统硬件设计基于高性能双核微控制器ESP32-S3进行，使用总线物理隔离以及DMA并行调度提高IO性能。ESP32 系列芯片集成无线通信能力并具有较丰富的外设接口，适合用于低成本、小体积物联网终端的硬件设计，因此本文选用 ESP32-S3 作为智能手表主控芯片具有一定合理性[10]。显示屏通过硬件高速SPI总线连接到ST7789P3驱动屏，设置80MHz最大传输速率并且绑定一个单独DMA通道，用于图形数据在后台进行低延迟异步传输，减少CPU绘图负担；大容量存储设备给SD卡分配一条独立物理SPI总线，在40MHz速度下结合FatFS文件系统保证音频文件流畅读取；低速外设将CST816T触摸屏以及SD3078 RTC模块挂在I2C总线上，利用中断方式代替高耗电轮询。电源部分由一块专门锂电池管理和一个3.3V电压调节器组成，保证系统在多种操作同时运行以及无线通信时保持稳定的3.3V电源供应并且具有较长寿命。

2.3.2.软件整体框架

为了避免复杂的视频解码以及SD卡读取造成阻塞对系统流畅产生影响，系统不再使用传统的单线程“大循环”，而是使用FreeRTOS来实现多任务处理。有文献表明，FreeRTOS可以在资源有限的微控制器上实现任务调度并且可以配合MQTT完成物联网的数据通讯，非常适合复杂的嵌入式设备上的多任务协作开发软件[7].部分被划分为若干个不同的任务(Task)，如UI主线程具有较高的优先级，只负责LVGL的界面更新以及触控事件下发；网络线程用于后台管理WiFi连接情况等；另外还定义了一个专门的“存储工作线程”，用于从SD卡中读取电子书及视频帧等工作，并利用FreeRTOS中的队列和信号量保证与解码任务之间的安全通信从而保证音视频流畅性。

如图2-1所示，为了使代码具有良好的内聚性和弱耦合性，系统软件采用分层结构及模块化设计思想。而在嵌入式软件开发中一般会采用平台化、分层式和构件化设计理念，以便减少底层硬件对上层业务造成影响以及便于后续系统维护及增强其可扩展性[8]。整个程序大致分为三层，而且使用异步消息队列把业务状态与UI分离：

(1) 第一层：硬件层

作为系统的物理基础，它控制底层外设，向上为应用层提供解耦、标准的驱动接口。主要包括六个最基本的硬件系统：触摸屏幕相关硬件（显示芯片、触控芯片、背光等）、存储硬件（SD卡、外置Flash芯片等）、总线与接口（SPI、I2C、GPIO底层外设等）、时钟与时序相关硬件（RTC实时时钟、系统定时器等）、网络相关硬件（Wi-Fi无线射频链路）、电源及唤醒相关硬件（供电、休眠唤醒等）。

(2)第二层：应用层

作为系统的“后端”，是进行各种重要工作的地方，这里包含了设备的基本系统服务（例如时间同步、天气获取、Wi-Fi连接及状态管理、本地与云端OTA升级、功耗控制等），也包括多媒体以及娱乐相关的代码实现（如小说显示、图片解码、MJPEG视频解码、游戏的核心逻辑等）。同时还有异步队列用于文件扫描以及任务间的相互协作。本层的所有应用模块在完成对数据的操作或者状态发生变化之后不能直接调用图形库的方法，而应将状态变更的通知封装后放入异步传递的消息队列上报。

(3)第三层：显示层

由于它是整个系统“前端”的视觉及交互入口，基于LVGL等图形引擎在单独的一个显示线程上运行，在这个线程的主要工作就是不断从消息队列中获取应用程序的状态变化并进行快速的UI重绘。

显示层内部逻辑包括界面路由交互、UI页面渲染以及底层显示输出深度三者联动：系统先统一截获用户触摸或按键等操作，以便完成各个不同页面间跳转（例如主表盘、系统设置、信息展示等）、状态传递及事件触发；之后利用UI渲染集合把所有业务数据都展示出来；最后由显示引擎负责组织整个图形绘制过程以及控制屏幕重绘时机，保证后端异步变化能及时准确地反映到前端屏幕上并且无撕裂、一致性良好。

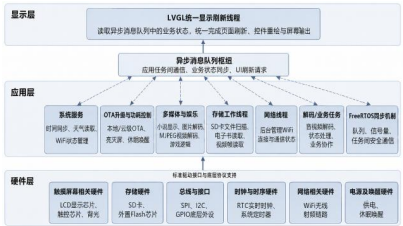


图2-1 ESP32-S3低功耗手表整体架构

2.3.3 软件局部框架

在完成整个系统的软硬件设计之后，在此对软件系统中的主要部分进行拆分并进行简要介绍。各个部分都在FreeRTOS的任务管理下协同工作并通过消息队列、事件回调以及驱动程序交互来传递信息及更新状态等操作。这样的模块化的设计可以让该系统在硬件资源较少的情况下也能保持较好的多任务执行能力同时也方便日后添加新的功能或者对其进行调试等。

电源管理与低功耗总体策略

如图2-2所示，系统实现了完善的低功耗状态管理。系统在一定时间内未检测到任何触控操作的情况下，系统将关闭背光，同时停止高能耗的WiFi以及音频视频等任务，让ESP32S3进入Light Sleep模式；而在硬件方面，采用具有自身休眠功能的CST816T触控IC，在有手指触摸或者移动显示屏时，CST816T会在很短时间内拉低连接在主控芯片上的IO口触发外部中断，从而使ESP32S3立刻苏醒并恢复正常工作速度，做到“待机极省电、交互即刻反馈”。

而手动长按确认键三秒，系统将进入另一种低功耗模式Deep sleep模式，除了会进行上一种进入低功耗前的操作，还会使整个主控芯片进入一个更低功耗的状况，并且该低功耗模式只能通过确认键唤醒无法通过触控屏幕实现开机操作



图2-2 低功耗状态进出示意图

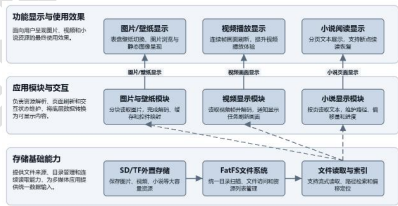


图2-3 SD 卡多媒体能力与系统架构图

SD卡综合应用与多媒体模块总体设计

如图2-3所示，系统的大容量存储使用SD卡挂载FATFS文件系统，采用简单的文件读取及索引方式支持图片壁纸、视频播放和小说阅读三种功能模块，在显示交互层面实现图片快速加载并映射、MJPEG视频异步缓存及连续解码、TXT文档按需分页及NVS保存偏移量。可实现在内存受限的情况下满足大量多媒体内容访问需求的同时保证界面平滑以及使用流畅性。

时钟管理与时间协同模块总体设计

如图2-4所示，系统采用“时间输入-RTC更新-显示输出”三层配合结构，使用Wi-Fi授时以及人工设置两种方式作为时间源输入，在连接互WIFI的情况下使用NTP获取时间后保存至SD3078进行时间更新，当处于断网下时由RTC定时更新时间库，以此达到网络高精度同步以及本地高可靠守时相互补充构成闭合回路，使整个系统始终具有准确、连续以及可用的时间。

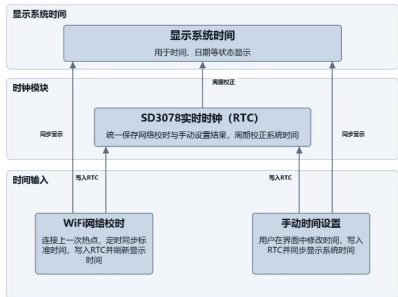


图2-4 时钟管理与时间协同模块设计

固件升级与维护模块总体设计

如图2-5所示，系统使用A/B双分区进行固件更新，在升级过程中不会直接覆盖当前运行系统，而是先将新固件写入另一个分区；升级包来源于云端以及SD卡，但是操作方式是相同的，即先下载或者从SD卡中读取，然后写入并且验证其正确性，在确定无误之后再切换为目标分区并重启以进入新版本；设备启动之后还会进一步检测新固件是否正常可靠，如果存在问题就回退到旧版，这样既保证能够被更新又能尽可能减少由于升级导致的问题给设备带来的不便。

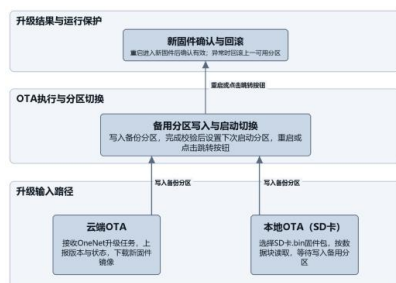


图2-5 OTA与本地升级流程图

2.4 本章小结

本章主要完成低功耗触控智能手表的需求分析及总体方案设计。根据智能手表的应用环境，对系统的人机交互、屏幕显示、文件存储、网络通信、时间同步、低功耗工作状态以及后期维护等基本要求进行讨论；再根据课题的要求，对比不同主控芯片、显示触摸模块、存储器、实时时钟模块、电源管理单元及编程平台的选择，在此基础上选择以ESP32-S3R8作为主控制器，并搭配LVGL图形界面、FreeRTOS实时操作系统、SD卡存储、RTC时钟以及WiFi通信模块搭建整个系统。以上述分析为基础，便于之后的工作展开。

3.硬件设计

3.1 ESP32-S3 最小系统电路设计

本设计的主控芯片并未采用市面上封装好的现成模组，而是为了实现更高的集成度与更灵活的空间布局，直接采用芯片级的ESP32S3R8搭配外部16M Flash、40MHz晶振与下载控制电路搭建最小系统。ESP32S3最小系统电路主要由核心主控、外部存储（Flash）、时钟晶振电路、复位电路与下载控制电路五部分组成。

3.1.1 ESP32S3R8主控芯片

ESP32-S3是乐鑫科技针对物联网、智能人机交互以及边缘计算所设计的一款高性能微控制器。它拥有双核32位处理器，最高主频可达到240MHz并且具有较多外围设备接口，可以进行图形显示、数据处理和无线通信等工作。已有物联网节点设计研究表明，ESP32具备WiFi通信、外设扩展和小型终端部署等优势，适合用于资源受限的物联网硬件节点设计[17]。相关的ESP32-S3嵌入式图像采集系统研究也证明，这种主控可以很好地满足图像数据处理及嵌入式边缘应用的要求[9]。由于需要支持大量多媒体数据处理、高分辨率图形渲染以及大量连续帧数据传输的需求，在此项目中我们使用了ESP32-S3R8作为主要控制芯片，其中“R8”表示芯片集成了8MB八线高速伪静态随机存取存储器，在比外部添加方式上，集成就能减少线长带来的影响，提高系统的鲁棒性和可靠性，同时由于是八线并行连接方式所以其有较大的带宽来读写大体量的数据，解决了传统的单片机片内静态随机存取存储器空间小不能满足复杂的显示和多媒体处理的问题。

ESP32-S3R8采用分层存储硬件架构，在片内集成高速SRAM，具有很低读写延时，可以被外设直接访问并进行快速传输，可用作显示链路或者数据传输高速缓存，另外片内合封8Mbit 8线扩展存储器主要是因为其大容量，用来存放图像数据、显示帧缓冲、多媒体缓存等大数据量信息，在这两个存储器之间通过片内高速总线连接起来，形成一个既有速度又有容量还有可靠性的整体存储系统。

基于以上硬件结构，本系统主控部分也是模块化、高可靠的实现方式，电源以及射频、时钟、外设等都进行了相应的优化处理。电源部分只有一路3.3V供电，但是对各个电源域都可以提供稳定的电压，而且在电源入口处有多个不同容量的电容组成的多级滤波器，还有滤波电感，可以有效去除电源上的纹波以及高频干扰，使芯片核心、外设以及射频部分得到良好的供电。

射频部分有一个天线、一个匹配电感以及电容构成的2.4GHz射频匹配网络，可以让信号在整个路径中的阻抗一致，从而提高无线通信的效果，增加无线通信的距离。

时钟部分有外部晶振接口可以接匹配电感，再加内部振荡器可以给整个芯片提供准确的主时钟，保证CPU和外围设备工作的正常进行；而且实时时钟域的供电也有单独的滤波电容来保证在低功耗下工作的准确性。

外设部分，芯片的所有GPIO都是可以引出的，方便以后的功能扩展；专门的SPI也可以引出用于控制高速显示或者存储设备，它对应的供电部分也有去耦电容，保证高速通信的信号质量良好。另外还留有调试接口，可以在编程时进行调试以及程序下载，便于开发和后期维护。

相关嵌入式图像采集与显示系统研究表明，微控制器平台能够完成图像数据采集、LCD 实时显示以及外部存储等功能，为本文实现图片显示和多媒体播放功能提供了参考[11]。从显示及交互硬件接口方面来看，该芯片集成了一个高效率的SPI，可以工作在较高的时钟下，从而可以与LCD驱动器良好通信并且通过专用DMA控制器，实现无需CPU介入的数据传输，提高画面刷新率的同时也减少了CPU负担，提高了整个系统的响应速度；相关TFT-LCD显示接口研究表明，显示总线结构、控制时序和底层驱动方式会直接影响屏幕刷新效率、引脚资源占用和系统可靠性[18]。而且该片还具有标准的高速串行通信接口IIC，可以可靠地连接到电容触摸屏控制器以获得及时准确的触摸屏位置信息以及迅速做出反应。总体来说，本设计使用一片ESP32-S3R8加上外接Flash、40MHz晶振、复位电路和下载控制电路组成最简系统。相关的ESP32物联网设备的研究表明，电源供电情况、时钟源、下载连接端口以及无线通信连接情况都会对物联网设备造成很大影响[10]。

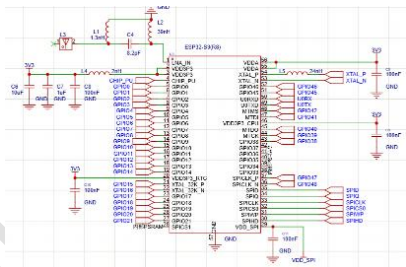


图3-1 ESP32S3R8主控芯片

3.1.2外部Flash存储电路

本系统由于主控芯片非易失性存储空间较小，在可控电路基础上外扩一片大容量串行Flash芯片，选用型号为W25Q128的16MB Flash作为外部存储设备，图3-2为其原理图。W25Q128 等外部 Flash 存储器具有容量较大、接口简单和非易失保存等特点，适合用于嵌入式系统中存放字体、图片、配置参数及固件相关数据[19]。此Flash支持高速四线串行外设接口传输方式，使用专门的时钟、片选、数据输入/输出等信号线与主控芯片上的高速外设接口相连接，接口电源域单独供电并且加有过滤电路以保证高速传输过程中的信号质量和稳定性。已有嵌入式存储系统研究表明，外扩串行Flash能够为程序存储、资源文件保存和系统参数记录提供可靠的非易失性存储空间，是小型嵌入式系统中常见的存储扩展方式[19]。从硬件连接来看，Flash的控制信号以及数据信号都直接连接到主控芯片相应的外设引脚上，构成一条完整的高速串行存储总线，在高频率时钟下可以进行连续的数据读写操作，为用户提供大量且安全可靠的非易失性存储空间来存放程序代码、固件、静态资源等。

从硬件结构来看，这个Flash与主控芯片的高速串行外设接口完全契合，四线并行的方式相比传统的单线接口大大提高了数据传输速率，可以很好地应对大量程序固件、图形资源等数据的大批量读取或写入的需求，而且由于其拥有较宽的工作电压范围以及较长的数据保存时间，可以在长时间内可靠地存储系统的设置信息、固件备份以及静态资源等关键数据，从而保障嵌入式系统的正常运行以及功能拓展。

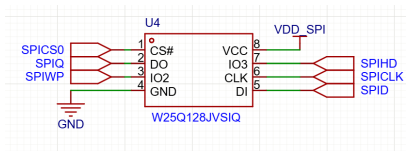


图3-2 外部16M Flash

3.1.3 40MHz晶振时钟电路

系统的主时钟基准是一个40MHz无源晶体振荡器，它是整个数字系统频率的标准。这个晶振连接到主控芯片上的时钟输入/输出端口，两侧并联适当大小的去耦电容，以改善晶振的负载谐振特性，减少高频噪声的影响，保证晶振正常工作并且产生准确频率。主控芯片中的高频锁相环根据这个40MHz的基本时钟进行倍频或者分频得到CPU核心使用的快速时钟、各个外设总线时钟、无线通信部分所用的高精度射频时钟等，从而给整个嵌入式系统提供稳定

可靠的时序基准，是使系统中所有的数字器件都能正常工作的基础硬件。

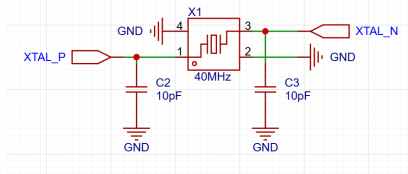


图3-3 40M晶振

3.1.4 下载控制与复位电路

本系统固件下载、调试以及供电接口均为标准Type-C物理接口，接口电路经过统一规划以保证其良好兼容性和可靠性，在接口上配置通道引脚也都是用一个固定的电阻来区分不同的设备类型，可以连接所有常见线材以及电源设备进行烧写操作；差分数据线连接到主控MCU USB接口上，可实现USB通信，用于固件下载、调试时的数据传输及供电。相关嵌入式调试接口设计研究表明，将供电、程序下载和调试通信接口进行统一规划，有助于降低硬件复杂度，并提升设备开发和维护过程中的便利性[20]。

在复位以及启动配置电路中，系统为复位引脚连接一个上拉电阻和一个滤波电容实现硬件复位；而启动配置引脚通过上拉电阻设定一个默认电平值并且保留一个可由外部进行干预的路径，可以利用硬件手段改变这个引脚上的电平来选择不同的启动模式，以满足固件烧写及调试的需求。这样就使得整个项目的开发更加方便快捷，不需要使用额外的桥接芯片就可以直接使用Type-C接口进行固件下载以及调试，同时也保证了系统的鲁棒性和易用性。

图3-4 TYPE_C接口	图3-5 复位与下载电路
---------------	--------------

3.2显示与触控模组电路设计

本设计使用一块1.83英寸显示触摸一体化模组，引脚定义见图3-6，整体示意图见图3-7。其内部集成ST7789P3显示驱动电路和CST816T电容触摸检测电路，分辨率为240×284，具有高亮度、大视角和全方向可视等特点。相关图像采集与显示系统研究表明，TFT-LCD能够在嵌入式平台上完成实时图像显示，适合用于低成本、小尺寸人机交互终端[18]

图3-6 显示屏FPC转接板	图3-7 屏幕整体示意图
----------------	--------------

3.2.1 LCD 显示接口电路（ST7789P3）

显示部分使用SPI硬件接口进行通信，由于其具有较高的传输速率以及较少占用硬件引脚的特点，使得整个系统更加简单明了。模组片选、命令/数据选择、串行时钟、串行数据均通过主控芯片上的SPI外设硬件引脚连接，而复位信号由主控芯片的普通IO口负责，用于上电初始化以及异常复位。背光控制信号接入主控芯片带有脉冲宽度调制输出功能的硬件引脚，可以用来设置屏幕的亮度。为了满足高分辨率下的良好刷新率，SPI有最大的80MHz时钟速度，在此基础上配合主控芯片内部的直接内存访问控制器，可以把显示缓冲区中的数据快速传输至显示驱动芯片中，大大减轻CPU负担并且提高传输速率，从而保证良好的用户体验。

3.2.2 触控接口电路（CST816T）

触控部分使用CST816T电容式触控检测芯片，该芯片具有低功耗工作模式，在待机情况下消耗很小的能量，适用于对功耗要求高的便携设备。触控通信使用了I2C硬件接口，时钟线和数据线上都连接有外接上拉电阻，保证高频率传输下良好的稳定性和可靠性。此款触控芯片还拥有硬件中断功能，其中断引脚与主控芯片的外部中断引脚相连接，可作为硬件唤醒路径。当设备处于低功耗待机模式时，触控芯片仍然以低功耗方式进行监测，一旦检测到有触摸操作发生，就会产生一个硬件中断信号给主控芯片的外部中断引脚，从而使得整个系统被迅速唤醒。这个过程不需要CPU不断地进行查询，可以大幅度减少设备的待机能耗的同时还能提高触控响应的速度以及灵敏度。相关嵌入式触摸屏驱动研究指出，采用I2C通信结合中断触发的方式，能够简化触摸数据读取和事件上报流程，适合应用于嵌入式人机交互输入设备[12]。

3.3 独立 RTC 时钟电路设计

为了解决主控芯片内部实时时钟在系统主电源掉电后不能正常工作问题，在此设计中采用SD3078高精度独立RTC芯片作为外部时间源[13]。SD3078内置一个32.768kHz温补晶振以及温度补偿电路，可以根据周围环境的变化

来对振荡频率进行校正从而降低由于普通晶振产生的温漂而造成的影响。此外它还具有低功耗的特点，在系统处于待机或者低功耗模式下也可以完成基本的时间守恒任务，从而为表盘上的时间显示、网络对时及日志的保存提供可靠保障。

从硬件接口看，SD3078使用标准I2C通信协议，时钟线以及数据线都由一个10kΩ上拉电阻连接到3.3V电源形成标准开漏结构，保证通信可靠的同时又非常节能。有文献指出，I2C总线仅需SDA和SCL两条信号线即可实现器件间通信，常用于连接EEPROM、实时时钟、A/D或D/A转换器以及温度传感器等外围器件，因此适合用于本文中RTC、触摸芯片等低速模块的数据传输错误!未找到引用源。。而在本文设计过程中，SD3078与微控制器之间通信线只连到微控制器相应I2C端口，以实现稳定可靠地发送时钟和数据，同时由于其本身带有少量内存，所以也可以用来存放一些必要的少量信息而不需另外增加存储单元，这样就使得整个系统的复杂性和成本降低。

图3-8 RTC时钟电路

3.4外部大容量存储电路设计 (Micro SD 卡)

SD卡尺寸小、容量大、非易失性和兼容性好，在嵌入式数据记录以及文件存储方面得到广泛应用[15]。由于系统需要较大容量本地存储以供使用，因此在该设计中加入Micro SD卡接口，给用户保存照片、视频、TXT文本文件及固件更新文件等提供可扩展非易失性存储空间。

本系统使用SPI串行通信方式与Micro SD卡进行通信，只需少量的时钟、片选、数据输入以及数据输出等引脚即可减少占用主控GPIO资源，卡座的重要信号线与主控芯片的相应SPI引脚对接构成一条完整的串行存储传输路径，可以实现连续文件读写以及多媒体数据读取，已有SD卡文件系统的相关研究也表明，SPI通信配合FAT类文件系统可以在嵌入式系统上有效实现数据读写、文件操作等功能[16]。

为了使存储接口在高速读写时正常工作，电路中还对电源以及信号线进行优化。卡座使用稳定3.3V供电，在接近供电引脚位置放置去耦电容防止读写过程中出现电源纹波问题。卡检测引脚与主板普通IO相连，根据其高低电平可得知是否有存储卡被插入或者移除，有利于提高整个存储装置实用性及鲁棒性。

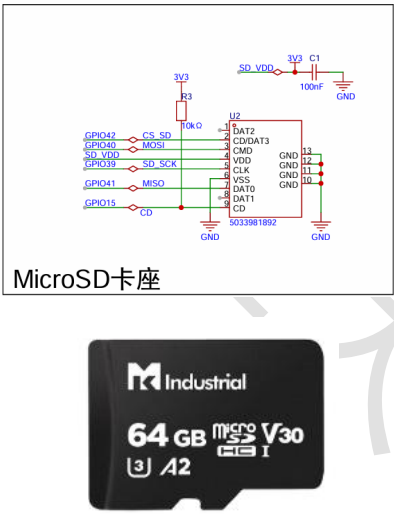


图3-9 SD卡座

图3-10 SD卡

3.5 电源管理与充电电路设计

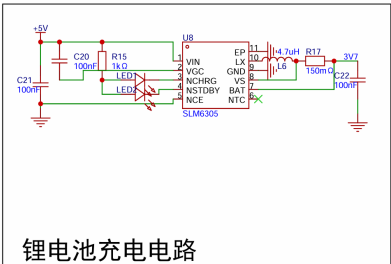
因为智能手表体积较小并且各个功能单元对于供电稳定性有较高的要求，所以本文采用线性充电IC以及低压差线性稳压器实现电源设计，简化电路的同时保证供电稳定。电源主要是由锂电池充电电路以及3.3V降压稳压电路构成，分别为电池充电提供能量以及为整个系统供电。

如图3-11与3-12所示，在3.3V降压稳压电路中，主控芯片、显示模块、存储器等外设都需要稳定可靠的3.3V电源，但是单节锂电池电压会随着电量变化，在大约3.0V到4.2V之间变化。所以本设计采用ME6217系列低压差线性稳压器作为主要降压元件，将锂电池电压转化为稳定的3.3V输出。该元件有较强的带载能力，在大电流情况下也能给主控芯片以及外围设备提供足够的瞬态电流。同时，稳压芯片的输入连接锂电池供电线路，输出连接整个系统的3.3V，而在输入与输出两端分别连接滤波电容与高频去耦电容，用于去除低频纹波以及高频噪声，使整个系统供电

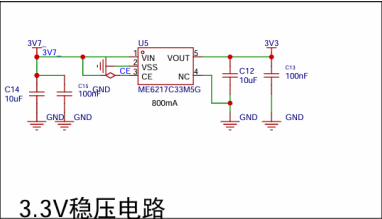
稳定可靠。

锂电池充电模块用于把外接5V电源转化为给单节锂电池充电所需要电压，保证安全充电，充电芯片接线方式是其输入接外接5V电源，输出接锂电池接口，在此基础上增加一个限流电阻、一个发光二极管以及一个状态引脚来形成一个硬件充电指示电路，当有外接电源并且开始充电之后，这个状态引脚就会被拉低，灯亮起，而当电池充满之后，这个状态引脚就会变成高阻态，灯会灭掉，所以不需要软件就可以知道当前充电情况，而且充电芯片内嵌有过流、过压、温控保护措施，可以避免因为过度充电或者短路造成安全隐患。

从整个供电路径方面看，外接电源接入既可以给锂电池充电也可以经过一个稳压电路给整个设备供电，在没有外接电源的情况下，锂电池通过一个稳压芯片输出稳定的3.3V电压用于给整个电路供电。这个供电方式简单方便、供电稳定且充电安全可靠，有利于保证智能手表的正常运转。



3-11 3.3V稳压电路



3-12 锂电池充电电路

3.6 本章小结

本章对低功耗触控智能手表的硬件系统进行实现。整体硬件电路如图3-13与3-14，硬件主要包括基于ESP32-S3R8主控最小系统、LCD显示接口、CST816T触摸接口、W25Q128外部Flash、SD卡存储模块、SD3078实时时钟模块、供电及充电电路、下载调试接口等组成。其中，显示模块利用SPI总线与主控通信，负责图像传输以及屏幕显示；触摸模块利用I2C总线获取触摸信息；SD卡、外部Flash分别保存照片、字库、文件及系统所需资源；RTC模块保证系统时间持续；而供电及充电电路保障整个设备正常工作并且低功耗。该硬件是后续软件开发的基础。

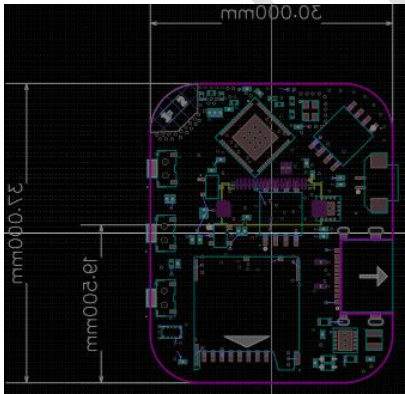


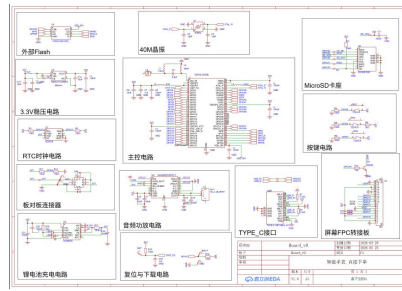
图3-13 整体PCB

图3-14 整体原理图

4.软件设计

本章主要介绍智能手表系统软件架构设计、多任务协作机制以及各个功能模块实现原理。系统基于实时操作系统开发具有良好人机交互性和协同性的应用程序。

本项目底层软件基于乐鑫提供的物联网集成开发框架ESP-IDF进行编写，此框架提供了一套完整的交叉编译工具



以及底层硬件抽象层接口，在SMP（对称多处理）方面也进行了良好的适配工作，可以充分发挥出ESP32-S3双核的优势。而在图形界面上，我们使用恩智浦（NXP）提供的GUI Guider进行设计，在GUI Guider中，设计人员可以在电脑上实现所见即所得的效果，实现控件的位置、层次关系以及动画效果设置等，最后会生成基于LVGL的C源文件代码，在工程上实现了UI视觉表现逻辑与底层业务核心算法分离的目的，大大缩短了智能穿戴设备复杂的交互界面的开发时间和维护难度。

4.1 低功耗手表系统整体架构与多任务协同调度设计

本系统软件结构是自底向上分层实现，在该基础上分为硬件层，应用层以及显示层。

硬件层是直接与ST7789屏幕、CST816T触摸屏、SD卡、Flash存储、RTC实时时钟、WiFi模块以及供电电路连接，利用SPI、I2C、GPIO等方式进行底层驱动，将繁琐的硬件时序、寄存器相关操作简化成一个统一的接口，由上层调用。

应用层是整个系统的主体功能部分，主要有时间控制、小说阅读、图片查看、视频播放、网络连接、天气查询以及OTA更新等。为防止复杂的操作卡住UI线程，本系统采用FreeRTOS实时操作系统，在其中把文件读取、视频解码、网络通信、时间同步等都做成不同的任务来执行并且使用消息队列进行各个任务之间的通信。有文献报道FreeRTOS可以把一个嵌入式应用程序拆分成许多任务并为其提供合适的处理器时间以及其他系统的资源以保证系统的实时性和可靠性[7]。

显示层基于LVGL图形库实现界面显示、页面跳转、动画重绘以及触摸事件处理。应用服务层在接收到新的数据或者发生的状态改变时不会直接对屏幕进行操作，而是把天气结果、小说分页、视频一帧一帧地展示以及OTA下载进度等内容都组织成一条消息传递给显示任务，在显示层负责更新相应的界面内容以及状态变化。这样就使得业务代码和UI分离，有利于多个任务同时进行屏幕刷新的安全性和一致性。

在显示输出上，LVGL实现文本、图片、控件以及动画等所有需要在屏幕上显示内容计算，在底层使用ST7789驱动程序利用SPI总线把像素数据传送到LCD上面，由于采用双缓冲及DMA传输方式使得CPU花费更少时间进行像素搬运工作，因此对于时钟显示、菜单切换、图片浏览以及视频播放时流畅性都有很大帮助。而对于嵌入式图形界面的研究表明，良好的GUI架构以及分层显示可以极大提升界面开发速度、实时性和系统的可维护性[18]。

在人机交互上，当电容触摸屏检测到点击或者滑动时，会发送一个中断给主控芯片读取触摸位置，然后用LVGL中的输入设备机制将其转化为点击、按压、滑动等逻辑事件，之后就会根据当前页面以及控件的状态进行页面跳转、列表滚动、小说翻页或者游戏控制等一系列的动作，从而完成从底层触摸到顶层显示的一次完整的交互过程。

4.2 横向架构的核心业务服务模块设计

横向架构是把复杂系统的各种功能拆分成独立的服务单元。每个应用层中的业务模块由FreeRTOS进行管理并发运行，并通过标准接口向底层硬件请求服务或者向上层显示提供准确、可信的信息。

4.2.1 SD卡综合应用与设计

如图4-1所示，SD卡是系统的高容量外部存储设备，用于存放图片、视频、TXT小说以及OTA固件更新文件等数据，在系统启动之后会先对SD卡进行初始化并加载FatFS文件系统，在指定位置查找相关资源文件并将其保存至一个文件列表中，在LVGL界面上用户点击某一项资源后，则按照不同文件类型进行相应的图片展示、视频播放还是文本阅读操作。

对于图片展示，在图片读取上，程序使用LVGL文件系统接口从SD卡中读取PNG或者JPEG格式的图片。由于ESP32-S3本身内存较小，不能一次性读入一张完整的图片，所以程序采取分段读入然后进行逐帧解码的方式，把解码后的小段图片更新到图片控件或表盘背景上，减少内存使用量。

在视频播放上，系统依据用户选定视频文件位置及类型决定播放方式，对压缩视频，程序逐帧读取并解码成可在

屏幕上显示内容，而原始像素视频则直接读取每一帧数据，在每一帧准备好之后，再让显示任务进行一次刷新即完成一帧显示从而达到连续播放目的。

对于TXT小说阅读，在打开文件之前，系统会读取历史阅读位置，在有记录的情况下，直接跳转到上次的位置。而在阅读中，以屏幕大小为单位进行分页显示；当用户翻页、退出阅读或者设备进入休眠状态时，系统会保存当前文件以及偏移量，以便下次阅读使用。

总体而言，SD卡相关功能整合成“文件扫描、路径解析、数据读取以及界面刷新”的过程。这样有利于减少图片、视频及文本显示子系统间的耦合关系，

也有利于多媒体文件在内存空间较小的嵌入式设备上良好运行。

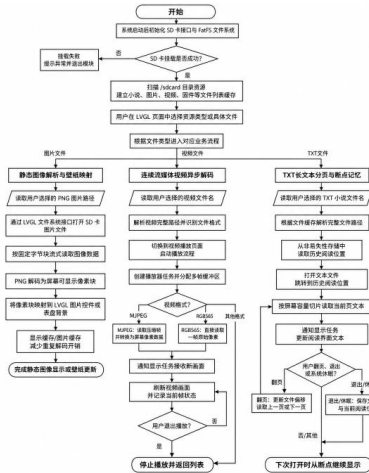


图4-1 SD卡综合应用与多媒体模块设计流程图

4.2.2 时钟管理与时间协同模块设计

时钟管理和时间协同模块用于维护系统的整体时间，在时钟显示、天气更新、日志记录以及定时任务中使用一致的时间源。如图4-2，当程序运行时，先对外部SD3078硬件RTC进行初始化并读取RTC中的时间来同步给ESP32系统时钟，在进入主界面之前设备具有有效的的时间。

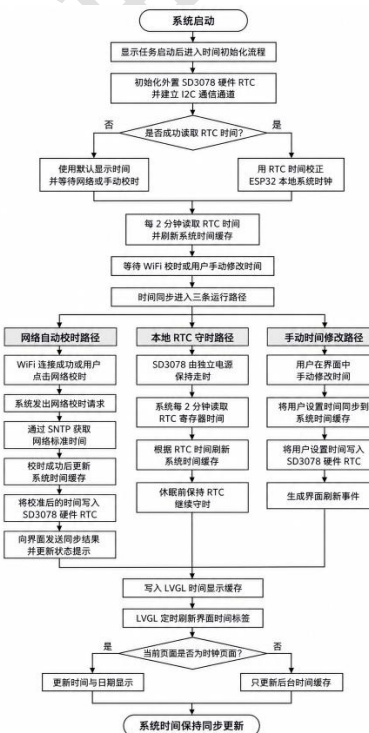


图4-2时钟管理与时间协同模块设计流程图

在进行网络校时的过程中，在WiFi连接成功或者用户主动发起同步操作之后，设备使用SNTP协议向授时服务器请求相应的时间信息并把校准过的时间保存到系统的缓存以及SD3078内，从而使软件中的时钟与时钟一致。

对于本地守时，在SD3078RTC负责离线情况下的时间保持工作。系统每隔一段时间读取一次RTC时间然后更新系统时间，减小长时间运行造成的时间误差；当系统关闭或者处于低功耗模式时，RTC仍然可以单独计时，在下次开机时提供给用户正确的时间。

从界面上看，显示层每隔一段时间读取同一时刻缓存并刷新时钟页面上时间与日期内容，在用户于时间设置页面自行调整时间之后，程序也会相应地对ESP32系统时钟以及外部RTC进行更新，使得这三种方式所设定的时间能够保持一致。

整体而言，系统使用“网络校准、本地守时、界面刷新”的方式工作，在网络连接状态下利用SNTP获取标准时间，在无网络情况下由外部RTC保证时的一致性，在界面上以统一的时间缓存进行更新来提升不同情况下的时间准确性。

4.2.3 OTA远程升级与固件维护模块设计

OTA升级以及固件维护是用于对系统的固件进行远程更新以及本地维护的功能。如图4-3所示，系统使用A/B分区方式进行升级，当前固件处于一个分区中，而新的固件则被写入另一个非活动分区中，所以在升级期间不会直接影响正在使用的固件，如果升级失败也可以回退到之前可用的固件版本。

在云端OTA升级中，在连接上云端之后获取升级指令并返回当前固件版本信息。在检测到有新版本时，建立固件下载通道，把新固件镜像写入一个备用分区，在下载完成之后，对镜像进行校验，校验成功则提示用户是否进行切换，把备用分区设为下次启动分区，然后重启到新固件。

对于本地升级，系统提供从SD卡中选取.bin固件包进行离线升级，在选择升级文件之后，系统获取该文件大小，擦除非活动升级分区，并以数据块方式读取固件内容写入备用分区，在写入的同时显示进度条，当写入以及校验完成时，系统提示用户升级成功并询问是否继续下一步骤即更换为新分区然后重启设备。

对于启动确认以及回滚保护，在设备重启进入新固件之后，会检查新固件能否正常工作。如果可以正常工作，则认为当前固件有效；如果不能正常工作，则进行回滚保护，回到之前一个可正常启动分区启动。这样就减少了由于固件升级失败使设备不能启动的可能性，增强了系统的可维护性。

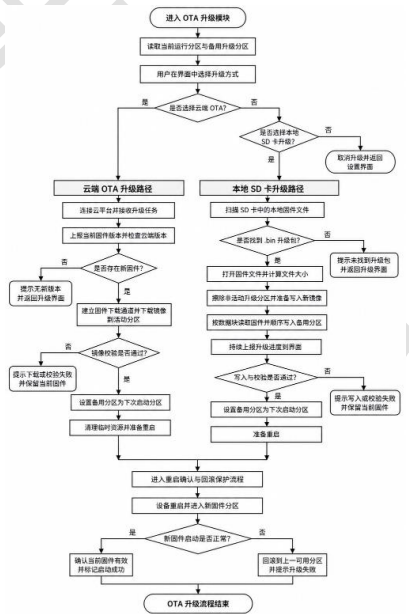


图4-3 OTA远程升级与固件维护模块设计流程图

4.2.4 开源休闲游戏扩展设计

为了让穿戴设备更有趣味性和人机交互体验，在该系统中使用LVGL图形库开发了贪吃蛇、记忆翻牌、像素飞鸟以及2048四个小游戏。这些小游戏都可以通过一个相同的游戏入口进入主菜单，在主菜单中点击对应的游戏就可以进入该游戏所在的游戏界面玩游戏，以后可以在此基础上增加更多的游戏模块。

每个游戏模块都有自己的运行状态，例如角色位置，得分，卡牌状态或矩阵中的数值等；用户触控、拖动等操作作为输入事件影响这些状态的变化；同时界面根据当前状态进行重新绘制相应的画面以及得分等内容。这种方法简单高效并且占用较少资源，在小尺寸便携式设备上可顺畅运作。

(1)贪吃蛇网格运动与碰撞逻辑

贪吃蛇游戏是在一个二维网格上运行。每隔一定时间，程序会更新一次蛇头的位置，在此基础上，再由用户触摸的方向决定其方向，在每次移动后都要判断蛇头是否撞到屏幕边界还是撞到了自身，如果是则视为游戏结束；而如果它吃到食物，则蛇身会加长并且得分增加。

(2)记忆翻牌匹配逻辑

记忆翻牌游戏采用卡片矩阵的形式排列，每一张卡片都有一个背面数字，在玩家点击两张卡片之后，程序会检查这两次点击是否是同一张卡片，如果是，则认为是配对成功并且一直保持翻开的状态；如果不是，则让玩家再选另一张卡片继续比较。该玩法简单易懂并且容易上手，在手机等小屏幕设备中适合作为消磨时间的小游戏。

(3)像素飞鸟单轴运动与障碍判定

像素飞鸟游戏采用单轴移动模式，在游戏中小鸟处于持续下落状态，玩家点击屏幕给予小鸟短暂向上速度使其飞行；障碍物向左运动，离开屏幕后又有新障碍物产生，在小鸟碰到障碍物前不会有任何得分，在小鸟躲避障碍物时才会有相应得分。

(4)2048矩阵位移与合并逻辑

2048游戏是基于数字矩阵实现，在系统检测到用户的手势之后，依据此手势的方向对待处理的一行或一列执行一系列操作：先压缩非零数字，接着如果有两个相邻且数值相同的数字，则进行两者之和运算，最后再将矩阵重新排序。若此次操作有所变化，系统就生成一个新数字方块并根据矩阵中的值改变其颜色、文字以及分数等。

4.2.5 系统多级低功耗电源调度机制设计

系统多级低功耗设置如图4-4，因为可穿戴设备电池电量较少，所以设计了系统能够自动进入轻休眠以及按键深休眠的方式来降低功耗。当系统运行时一直记录下用户上一次操作的时间并且每隔一段时间就去检测是否处于空闲状态；同时也会去检测确认键是否被按住并且按住较长一段时间。根据不同的情况让系统的状态变为轻休眠或者深休眠从而达到节约电量的效果同时也提供给用户更好的体验。

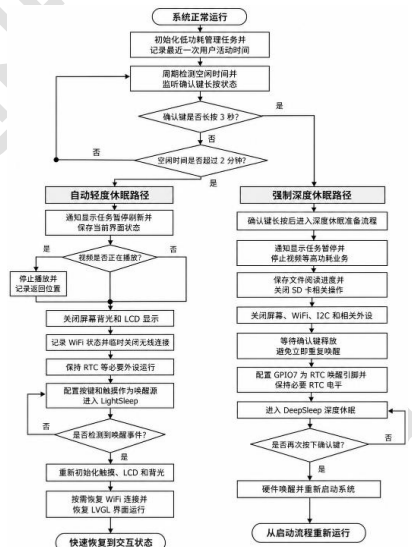


图4-4 多级休眠与动态唤醒流程图

在自动轻休眠模式下，当系统检测到大约两分钟没有人操作时，就先停止界面上的信息刷新让机器进入可以休眠的状态，在这段时间内如果有视频播放就会暂停视频并且记录当前播放位置，然后关闭屏幕背光及LCD的内容，同时断开Wi-Fi并且把一些外设设置成节能模式，最后用按键或者触摸来唤醒来回切换到LightSleep状态。

轻休眠特点是快醒，在用户点击或触摸唤醒设备时，系统重新初始化触摸、LCD、背光等，然后根据休眠前的情况决定是否恢复WiFi连接，然后再启动LVGL使其恢复正常工作，使设备迅速处于可用状态，适合短时间休眠，在节约电能的基础上也带给用户良好的体验。

在强制深休眠情况下，当用户持续摁下确认键超过3秒时，系统将执行深休眠前准备工作，在这一步骤中，系统停止刷新屏幕，停止播放视频等高功耗操作，保存阅读进度，关闭屏幕并且断开连接Wi-Fi、I2C、SD卡及其他外部设备等。之后将GPIO7作为RTC唤醒信号线保持一段时间再拉低电平之后进入DeepSleep模式。

深度休眠会关闭更多的系统资源，功耗更低，但是唤醒后需重新启动系统，当用户再次点击确认键后，硬件唤醒主控，系统重新开始启动的过程，在长时间未使用设备时，可以让手机处于更长时间的休眠状态。

4.3本章小结

本章介绍了低功耗触控智能手表的软件设计。软件方面采用ESP-IDF开发框架以及FreeRTOS实时操作系统实现模块化设计，将整个系统划分为底层驱动、功能任务和图形界面等几个模块。底层驱动主要是对显示屏、触摸屏、SD卡、RTC时钟等外围设备进行初始化及通信操作；而功能任务主要是实现屏幕更新、文件读取、网络连接、时间同步等；最后图形界面是利用LVGL和GUI Guider完成界面布局及交互逻辑的设计。使用任务间切换、消息传递和分层的方式来增强代码可读性和便于后期的功能添加或者问题排查。

5.低功耗手表系统综合调试

当系统的各个子模块与整体开发完毕之后，就进入到系统的综合集成及调试工作，在整个联调过程中，将论证与分析系统整体功能实现情况。由于该智能手表系统需要频繁进行图形界面绘制、复杂实时操作系统多任务调度、大量文件读写操作以及底层硬件通信等，因此在联调过程中也遇到了一些困难，在底层时序、内存管理、多线程并发以及人机交互等方面存在问题，下面将针对系统联调中遇到的主要问题进行分析并提出解决方案及改进措施。

5.1 硬件系统实物构建与整机组装集成

本系统的主控板是两层PCB结构，裸板上的布线密集，在底层布置了较多去耦电容焊接位置，以便保证ESP32-S3核心运行在高频率下良好的电源完整性和信号完整性。

5.1.1 核心控制板贴装与焊接工艺验证

由于本项目PCBA双面布线并且高密度贴片，为了保证细间距芯片引脚焊接可靠，元器件贴装方式为：“正面钢网大批量回流，背面热风手动辅助焊接”。

图5-1 PCB正面	图5-2 PCB背面
------------	------------

首先是将ESP32-S3主控芯片、16M Flash、SD3078与电容、电阻、电感，等焊盘用特制钢网涂覆适量锡膏，然后用镊子把元器件贴放在正确位置上，在恒温加热台上进行正面主要IC和阻容元件的高可靠焊接。在正面冷却固化之后，用热风枪对局部进行精确对流加热焊接，在整个焊接过程中小心处理细小引脚因锡膏溢出造成的短路问题，用吸锡带及时吸取过多的液态焊锡，使管脚分开且整齐。完成双面全板焊接后，用专用洗板水清洗板面上残余松香和助焊剂，消除因长期工作导致杂质造成微安级漏电流影响。再用万用表蜂鸣档检查底层电源网（VCC 和GND）阻抗，保证没有肉眼看不见微小短路存在。

图5-3 PCBA正面	图5-4 PCBA背面
-------------	-------------

5.1.2 机械结构建模与3D打印成型

为保护裸露电路及提高手表佩戴舒适度和按键手感，本低功耗手表系统使用SolidWorks对手表外壳进行建模。在表壳内腔依据PCB大小设置凹槽、卡槽以及固定孔位，以使得主控板、电池、屏幕及相关连接线可以牢固安装在表壳中，在佩戴或操作时不会发生移动。在结构设计中，外壳侧面因Type-C充电口、MicroSD卡槽及三个微动按键而开有孔位以便避开这些元件并且留有一定余量，使充电、读写以及按键操作正常进行。同时考虑接口位置与按键行程配合关系，使得外壳能够容纳并且防护内部电路而不妨碍外部连接或者操作。外壳是主机身以及按键帽分离设计。主体用黑色光敏树脂进行3D打印，三个实体按键帽分开制作再组装上去。这样可以避免一体打印造成的悬垂结构问题，同时让按键有足够活动范围，使得按压反馈更加明确，手感更好。

5-5 外壳3D仿真图	5-6 外壳3D实物图
-------------	-------------

5.1.3 整机集成与上电初步验证

在电路板硬件以及3D打印结构件都检查合格后，进行整体装配工作。装配过程中，先将焊接好的PCBA控制板依照所设计的方向放置到外壳底部，使其两侧Type-C接口、MicroSD卡槽以及按键的位置与外壳开孔相对应，在此基础上再安装三个单独按键盖子以保证按键能正常下按和复位。

然后连接LCD触摸屏模块，屏幕FPC软排线从外壳内预留缝隙进入控制板接口并塞入FPC座固定，以保证屏幕和主控板之间良好通讯。最后安装主板、屏幕以及按键后，整个机器已经基本成为一个封闭体。

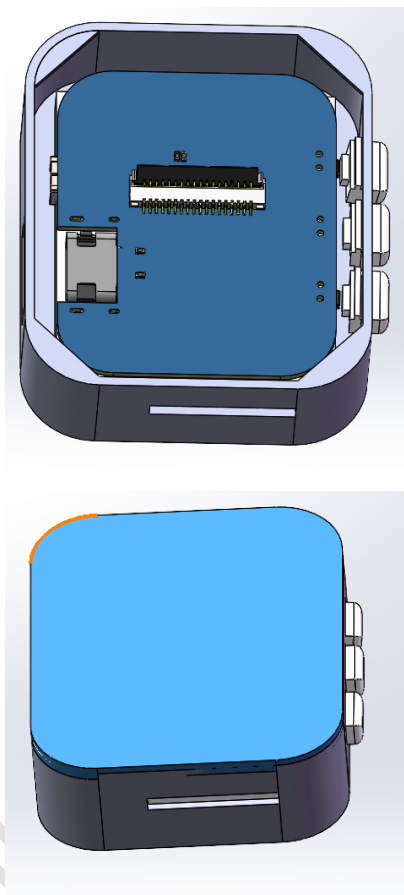


图5-7 手表3D示意图(不含屏幕)

图5-8手表3D示意图

第一次上电后，在下方Type-C接口处连接5V调试电源，系统可以正常运行并且加载固件，LCD屏点亮，有静态壁纸、硬件时钟以及日期等信息显示，则表明显示通道以及基本的时间读取是正常的。

点亮测试说明，微控制器最小系统、电源供电、LCD显示驱动、触摸屏连接及SD卡基本读写功能已实现，至此硬件组装工作结束，可进行下一步软件调试、功能测试及稳定性的检查等工作。



图5-9 手表实物展示

5.2 核心业务功能与整机交互体验评估

本章对系统核心功能进行全面测试，在UI手势操作、SD卡资源读取、多媒体播放、电子书阅读、WiFi连接与OTA升级等方面进行测试，为保证测试公平公正，在相同供电情况下，相同显示亮度以及相同固件版本下进行，主要统计各项功能成功次数、平均耗时以及出现的问题等，见表5.1，总体上系统在一般情况下可以正常运行各项主要功能，表明软件与硬件结合良好。

从交互角度上讲，主菜单高频滑动测试成功率为100%，触控响应时间低于50毫秒，表明LVGL事件以及页面跳转可以满足小尺寸触摸屏基本交互要求，在测试中一般性滑动、点击及返回等都能良好工作，快捷面板、主菜单与表盘页面之间切换也较为流畅，但是当频率非常高时由于屏幕刷新率以及总线带宽问题偶尔会有轻微卡顿现象发生，这并不影响正常使用但却是未来需要改进之处。

对于多媒体及文件读取，可以实现SD卡壁纸更换、MJPEG视频播放、电子书断点续读等操作。电子书阅读测试成功率为100%，再次打开文件可以回到之前的位置继续浏览，说明文件偏移的读写是可靠的。而SD卡壁纸更换成功率仅为83.3%，MJPEG视频播放率为93.3%，在连续快速更换大量图片或者高负载情况下，会有一段时间卡顿、无响应或者轻微掉帧现象发生，这主要是因为SD卡读取速度慢、图片解码所花时间和CPU资源有限有关。

在无线网络以及系统维护上，WiFi连接成功率为96.7%，平均连接时间为4.5秒，少量未能连接主要是由于周围环境影响，在重启之后可以恢复正常。OTA升级测试成功率为100%，无论是通过远程还是本地都可以实现对固件进行更新以及重启操作并且可以验证掉电情况下的回滚功能，证明该产品具有良好的可维护性和可靠性。

总体来说，在ESP32-S3上实现表盘交互、资源读取、多媒体播放、文本浏览、网络连接和固件更新等功能，基本符合课题需求。之后可以针对SD卡读取速度、图片解码速度、视频帧刷新以及大负载情况下的任务调度进行改进提升系统的平滑性和实用性。

测试功能项	成功率	平均响应	备注说明
主菜单高频滑动	30/30	< 50ms	交互稳定，高频滑动下偶发轻微画面撕裂
SD卡壁纸切换	25/30	约100ms	存在短暂解码显示延迟，少数情况下出现无响应
MJPEG视频播放	28/30	约25fps	高负载下轻微卡顿，偶有掉帧
电子书阅读	30/30	< 150ms	索引加载稳定，阅读位置定位准确
WiFi网络连接	29/30	约4.5s	极少数情况下受环境干扰导致连接超时，重新开启后可恢复
OTA升级（远程/本地）	10/10	约50s	本地升级包与OTA分别测试，断电中断后的回滚保护有效

表5-1 核心功能测试

5.3 系统功耗与多级休眠机制测试

智能穿戴设备对于续航有非常高的要求，在此为了检验系统的软硬件设计是否良好地进行电源管理而使用了可调节直流稳压电源（DPS-150）替代403035聚合物锂电池（标称容量为450mAh，标称电压为3.7V），在恒压3.7V下测量智能手表在不同的背光亮度、射频状况以及处于睡眠时的稳定电流以及最大功耗。

在系统运行状态下，LCD 屏幕背光与 Wi-Fi 射频模块是系统的两大核心耗电单元。测试过程中，以 10% 为步进调节屏幕亮度，并分别记录开启和关闭 Wi-Fi 时的功耗表现，具体测试数据如表 5-1 所示。

5.3.1 动态功耗测试结果分析

根据表5.2所示内容可知，系统功耗主要与屏幕背光亮度以及WiFi射频有关，在WiFi关闭情况下，系统电流随屏幕背光亮度增大而逐渐增大，当背光亮度从10%到100%，稳态电流从0.058A到0.163A，稳态功率从0.22W到0.62W，因此屏幕背光是在正常工作时影响功耗一个重要原因。

结合450mAh电池容量估算，在10%亮度、WiFi关闭的情况下，设备理论亮屏待机时间为约7.7小时；而在100%亮度、WiFi关闭情况下，则理论亮屏待机时间为约2.7小时。

WiFi打开的情况下，会有一个瞬时电流上升的过程。从实验来看，在射频连接期间是在原先亮屏的基础上再增加约0.065A到0.070A左右，主要是用于无线连接、热点连接以及网络同步等操作。这个高功耗的时间非常短暂，约3秒



图5-10 运行状态功耗测试数据统计现场

之后系统就会停止或者减少射频的相关工作，功耗也会随之下降。所以，通过只在需要的时候才打开WiFi并且在完成同步之后立即退出高功耗的连接状态的方法来节省由网络模块所引起的能耗。

表 5-2 系统多场景功耗测试数据统计表（恒压 3.7V）

运行状态	稳态电流 (A)	稳态功率 (W)
正常运行	0.058	0.22
正常运行	0.069	0.26
正常运行	0.080	0.30
正常运行	0.091	0.34
正常运行	0.103	0.38
正常运行	0.115	0.43

正常运行	0.126	0.47
正常运行	0.139	0.52
正常运行	0.153	0.57
正常运行	0.163	0.62
射频激活	增加约0.065~0.070（持续3秒后关闭）	相应激增
Light Sleep	0.008	0.01
Deep Sleep	0.006	0.01

5.3.2 多级休眠功耗异常分析与模式优势探讨

系统设有两级低功耗模式：约120秒未进行任何操作后，系统会自动进入LightSleep轻度休眠；约3秒长按确认键之后，系统会进入DeepSleep深度休眠。这两种模式都是为了解决不同时间段内使用的需要而设计，在保证较低能耗的基础上也考虑到了唤醒以及方便性问题。

根据表5-2所示，在WiFi关闭情况下，系统电流随着屏幕背光亮度增大而大幅上升。从10%到100%，稳态电流从0.058A变为0.163A，功率从0.22W变为了0.62W，因此可以认为屏幕背光是亮屏状态下主要耗电力的部分。而在WiFi正在连接的过程中，系统电流会比之前多出大约0.065A到0.070A左右，并且只维持3秒左右时间就断开，这说明射频部分在连接期间会有瞬时较大用电量。

进入休眠之后，消耗更少电能。在LightSleep模式中，屏幕背光、WiFi以及一部分外围设备被关闭，稳态电流大约为0.008A，在DeepSleep模式中，系统进一步减少更多的资源占用，只保留最基本的物理按键唤醒条件，稳态电流约0.006A，相比于10%亮度下工作时的0.058A，这两种休眠方式都大幅节省电量消耗，而且DeepSleep更低，适合长期不用或者存放运输的情况。

以450mAh电池容量为例，在100%亮度、关闭WiFi情况下可以使用约2.8小时；加上WiFi开启瞬间最大电流情况下的极限重载情况下可以使用约1.9小时。而在10%亮度、关闭WiFi的情况下正常亮屏状态下可以使用约7.7小时；LightSleep可以使用约56小时；DeepSleep可以使用约75小时。由此可见，该系统通过背光控制、WiFi按需启停以及两级休眠方式可以在亮屏交互、短暂时机和长期保存之间做到很好的折中。

5.4 系统现存不足与未来优化展望

虽然本智能手表系统在功能及功耗测试上基本满足要求，但是由于开发时间较短、手工制作样品生产工艺不良以及微控制器硬件资源不足等问题导致系统的多媒体流畅度不高、低功耗控制不佳、体积较大等问题。本文根据以上实验情况总结了当前样品所存在的主要问题并提出了一些解决方案。

5.4.1 软件渲染与总线带宽瓶颈分析

在多媒体显示以及高频界面交互时，有时会有轻微的画面撕裂或者短暂停顿的现象，在这种现象一般只发生在MJPEG视频播放、主菜单快速滑动以及大尺寸图片切换等高负载的情况下，由于系统已经使用了SPI-DMA以及缓存来提高显示速度，但是目前LCD刷新并没有硬件级别的同步信号，所以在图像解码、数据传输和LCD刷新之间的时间如果不能保持一致的话，就会造成一部分画面不同步的情况。

之后进一步优化可考虑引出LCD屏幕TE同步信号并连接到ESP32-S3外部中断，然后在LVGL刷新过程中加入垂直同步处理，让DMA刷屏尽量避开屏幕刷新区域。这可以在一定程度上减少快速移动的画面出现抖动的概率，在播放视频或者拖拽菜单等情况下可以使显示更加稳定。

另外，SD卡读取速度对多媒体体验也有很大影响。目前样品使用SPI方式进行SD卡操作，在读取大量图像文件、视频或者文本时由于总线频率较低以及PCB走线问题导致读取较慢。后续产品可以使用4-bit SDMMC接口并且合理布线时钟线和数据线以降低阻抗失配带来的影响从而增加外接存储读取带宽减少图片切换或者视频播放时延。

5.4.2 硬件电源与低功耗设计改进

对于时间保持，在目前的RTC芯片中仍然需要由主电池供电，没有专门备用电。如果主电池耗尽或者长时间无电的情况下，RTC可能会因为失电导致其内部的时间被清零。未来可以考虑加一个小容量法拉电容或者贴片式的可充电备用电，给RTC提供一个备用电源来保证在主电池断电之后还能有一段时间的基本走时功能。

低功耗方面，由表5.2可知，在LightSleep模式下系统电流约0.008A，在DeepSleep模式下约为0.006A，两者均远低于正常点亮屏幕工作时电流，但是从嵌入式低功耗设计来看，毫安级别休眠电流还是有优化空间。这主要是因为目前此款开发板在进入休眠后并未真正断开所有外围设备供电，一些GPIO、电源线路以及外设仍然可能存在少量泄漏电流。

之后可以在硬件电源拓扑上增加电源门控，比如在外设供电线上添加负载开关或者PMOS控制电路，在

DeepSleep状态下，通过RTC域GPIO控制外设供电断开，只保留必要的唤醒电路就可以，从而从物理上降低外围器件待机电流消耗，使得深睡模式更低。

这两种休眠方式都有各自的功能。LightSleep是在短时间内休眠，仍然保留触摸或按键唤醒的功能，能够快速进行人机交互；而DeepSleep是长时间休眠，依靠物理按键唤醒以减少误操作的可能性，在口袋或包中不易被触碰从而避免意外唤醒。后续工作不应仅仅是减少单一方面的功耗，还应、唤醒时间和续航等方面综合考虑整体设计的问题。

5.4.3 结构工程与整机空间优化

目前低功耗手表系统外壳通过3D打印制作而成，方便快捷进行结构设计，但由于打印精度以及材料强度问题，其外壳壁厚以及装配间隙都比较大，使得整个设备较厚较大。如果以后开发更高完成度样的产品时，提升结构精度并且减小壁厚，进而提高用户体验及手感。

内部空间使用上，屏幕FPC连接器以及屏幕FPC与主板之间存在一定的垂直距离，导致屏幕与PCB不能紧密接触，在下次PCB设计中可以将FPC座移到主板背面，在PCB上开一个过线孔，让屏幕排线从前面向后穿过主板再连接到背面接口，这样就可以节省一部分屏幕和主板之间空间，从而降低整机的厚度。

总体而言，目前系统已实现功能闭环以及样机测试，在显示同步、存储带宽、低功耗硬件断电及小型化等方面还有待提高，在接下来的工作中可以针对“更平滑显示、更低待机功耗、更小体积”进行改进以使系统更加稳定可靠并具有较好的商品化前景。

5.5 本章小结

本章主要针对低功耗触控智能手表系统进行调试以及功能实现情况。调试工作首先是检查系统基本工作环境是否正常，即是否正确焊接电路板、电源电压是否合适、下载接口是否完好等，之后再针对屏幕显示、触摸反馈、SD卡读写、RTC时间获取、WiFi连接、页面切换及进入睡眠模式和唤醒等操作进行了测试。经过测试发现该系统可以正常启动并进行简单的页面展示以及完成部分操作，说明整个系统设计具有一定的合理性。但是同时也发现了该系统对于大文件读取、页面翻页、媒体播放以及节电等问题上还存在问题需要优化。通过对本章测试可以为后面系统的改进提供一定参考。

。

6. 结论

本文以低功耗触控智能手表为研究对象，在需求分析的基础上进行硬件电路设计、软件架构搭建以及系统测试等工作。系统采用ESP32-S3R8作为主控芯片，利用LVGL图形库以及FreeRTOS多任务调度方式实现表盘展示、触摸操作、菜单跳转、快捷操作、天气查询、网络对时、音频播放、电子书浏览、休闲游戏、OTA更新以及低功耗待机等功能，满足了课题要求。

从硬件上看，系统实现主控最小系统、LCD显示及触摸屏接口、SD卡存储、SD3078实时时钟、电源管理和充电电路等部分，保证整个系统的正常工作；而从软件上看，系统采用分层结构以及消息队列的方式，把底层驱动、业务层与界面更新分离，便于后期开发与维护，在解决嵌入式设备内存不足的情况下，在图片显示、视频播放以及小说阅读等方面使用分块读取、缓存刷新以及断点保存的方法，使得SD卡中的多媒体资料可以在小尺寸设备上较好地运行。

从整体上看，在一般触摸操作、主界面切换、电子书断点续读、网络连接与OTA更新等功能上均良好；但在高分辨率图片翻页及MJPEG视频播放等重负载情况下，有一定程度解码延迟、轻微丢帧或者画面撕裂情况发生。对于续航能力而言，屏幕背光以及Wi-Fi模块是造成最大能耗的部分，而LightSleep和DeepSleep两种低功耗模式可以很好地满足设备在休眠状态下快速唤醒以及防止误碰的需求。

总体而言，本文研究证明了基于ESP32-S3微控制器开发一款简易智能手表是完全可行的。该系统具有良好的功能性、易用性、低功耗以及后续维护性。未来还可以针对屏幕刷新频率、SD卡总线带宽、RTC备用供电、电源门控及整个设备体积等问题进行改进来提高系统的可靠性和使用寿命等。

致谢

光阴似箭，四年的本科生活转瞬即逝，在这四年里，我学到了专业知识的同时也锻炼了自己的动手能力和分析问题的能力以及认真负责的工作态度，这四年给我留下了非常美好的回忆。最后我要感谢那些曾经在我学习上帮助过我的老师、同学、家人和朋友们。

首先，衷心感谢我的导师。从毕业设计选题、方案论证、系统架构设计以及软硬件实现到最后论文写作和修改过

程中,均得到老师的悉心关怀与指导,在此表示最深切谢意。老师认真负责的态度、渊博的知识、踏实的工作作风和追求卓越的精神对我影响很大,在今后的人生道路上将以此为鞭策勉励自己。

非常感激学校提供优越的学习条件、先进的实验室设备以及充足的文献资料,同时也十分感谢每一位授课老师无私奉献和耐心指导,在他们通俗易懂地讲解以及认真负责的态度下使我掌握了较为系统的专业知识,从而保证了我的毕业设计能够顺利完成;还要感谢实验室同学在我做项目期间给我的交流与协助,在一起讨论问题的过程中增进彼此之间的友谊也认识到合作的力量。

最后,我还要对我的家人表示最诚挚的谢意。一直以来,他们一直默默支持着我学习,在生活上对我悉心照顾、理解、体谅和支持,理解我的缺点、承受我的烦恼,是我求知生涯中最强有力的支撑、最温馨的避风港,也是我一直向前、无所畏惧的最大动力。

四年光阴似箭,毕业既是旅途的一个结束,也是新生活的开始。以后我会带着这份感谢,收获以及责任,在今后的日子里牢记初衷,一步一个脚印,不断努力学习,积极向上,认真做好每一件事,做一个称职的工程师,不辜负青春,不辜负学到的知识,也不辜负那些一直关心帮助我的人们

参考文献

毛彤,周开宇.可穿戴设备综合分析及建议[J].电信科学,2014,30(10):134-142.

王骥,郭海亮,任肖丽.基于蓝牙低功耗技术的智能健康手表系统[J].生物医学工程学杂志,2017,34(4):557-564.

张子财,张胜田,佟向坤.基于ESP32的农业定点灌溉节水系统[J].现代信息科技,2022,6(11):172-175.

储汇.基于低功耗蓝牙5.0的智能可穿戴设备设计研发[D].淮南:安徽理工大学,2022.

刘俊峰,乔有田.基于ESP32与微信小程序的智能家居控制系统[J].科技创新与应用,2024(25):41-44.

赵能卿,李加强.车载嵌入式软件A/B分区设计与开发[J].汽车电器,2025(02):73-76.

朱明辉,赵信广,尤星懿.基于FreeRTOS和MQTT的海洋监测网络框架[J].电子技术应用,2018,44(1):41-44.

周治国.浅谈嵌入式软件开发模式与软件架构[J].信息与电脑(理论版),2012(10).

梁小军,张松.基于ESP32-S3的嵌入式图像采集系统设计[J].电子制作,2023(20):35-38.

王刚,马晓东.基于ESP32的物联网终端硬件设计与实现[J].电子技术与软件工程,2021(18):101-103.

李慧敏,樊记明,杨笑.基于STM32和OV7670的图像采集与显示系统设计[J].传感器与微系统,2016,35(9):114-117.

刘林辉,张芬.Windows CE下触摸屏驱动实现的分析[J].现代电子技术,2010(12):91-93.

任峥峥,叶桦,孙晓洁.智能车载终端的GPRS多路连接通信的研究[J].测控技术,2013,32(1):33-36,40.

杨日容,林木峰,陆泉森.一种基于I2C总线驱动的锅炉温度测量系统设计[J].微型机与应用,2011(19).

苏义鑫,程敏,何力.基于AT89C52单片机的SD卡读写设计[J].世界电子元器件,2009(04).

刘人萍,汪涛. NIOS下实现存储速度可调的SD卡FAT文件系统[J]. 微型机与应用, 2017(08).

关世友,吴再群. 一种轻量级物联网节点管理软件设计及应用[J]. 物联网学报,2021,5(4):146-159.

黄克亚. 基于FSMC总线的嵌入式系统多显示终端驱动设计[J]. 液晶与显示,2022,37(6):718-725.

张超,刘凤玉. 基于W25Q128的嵌入式数据存储系统设计[J]. 电子设计工程,2018,26(15):166-169.

陈晓东,张磊. 基于USB接口的嵌入式系统在线调试设计[J]. 电子设计工程,2016,24(18):120-123.

说明

- 1、疑似AIGC概率越大，风险等级越高，代表AI生成可能性越大。
- 2、系统检测结果仅作为校方评价学生论文的参考依据之一，建议不作为最终评价依据。学生论文最终评价，以校方评价标准和评价结果为准。



关注微信公众号

官方网站:co.gocheck.cn

客服热线:400-699-3389

客服QQ:800113999